第六章　树和二叉树

6.1 树的类型定义

6.2 二叉树的类型定义

6.3 二叉树的存储结构

6.4 二叉树的遍历

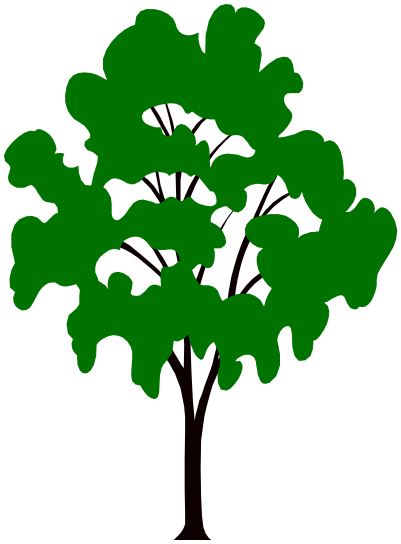
6.5 线索二叉树

6.6 树和森林的表示方法

6.7 树和森林的遍历

6.8 哈夫曼树与哈夫曼编码

6.1 树的类型定义



数据对象 D：

D是具有相同特性的数据元素的集合。

数据关系 R：

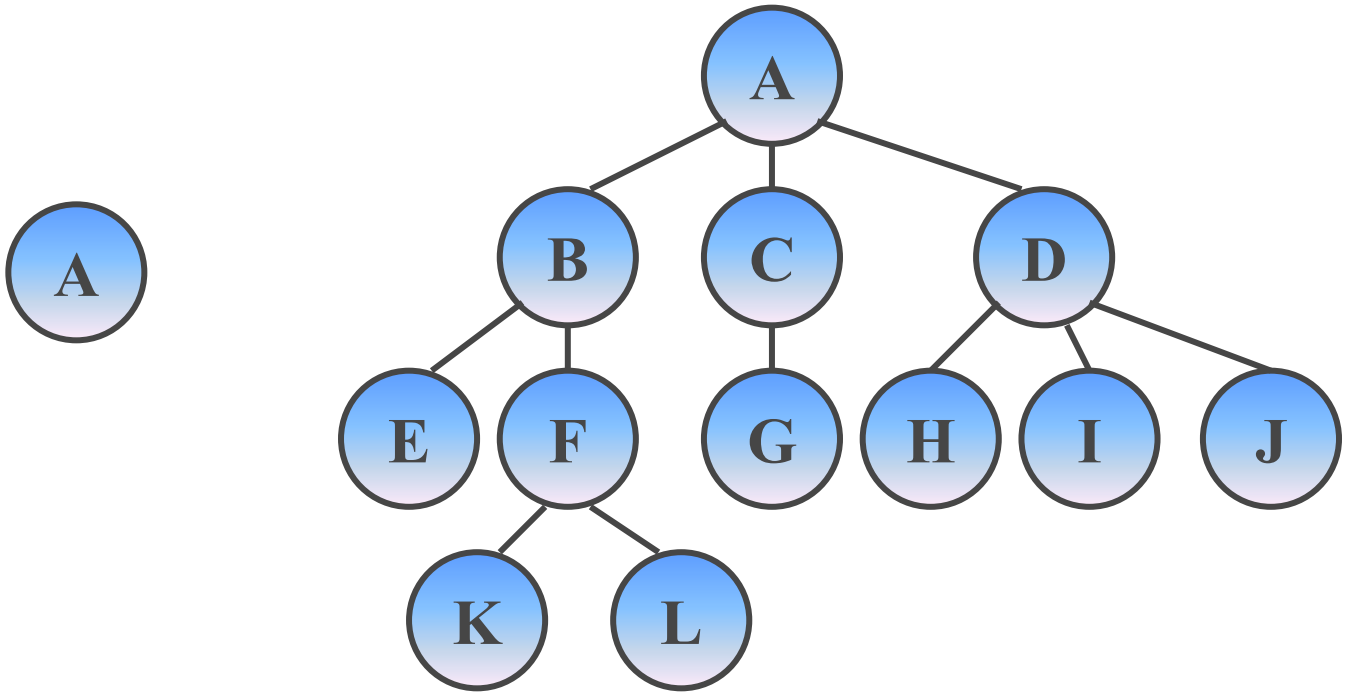
若D为空集，则称为空树 。

否则:

(1) 在D中存在唯一的称为根的数据元素root；

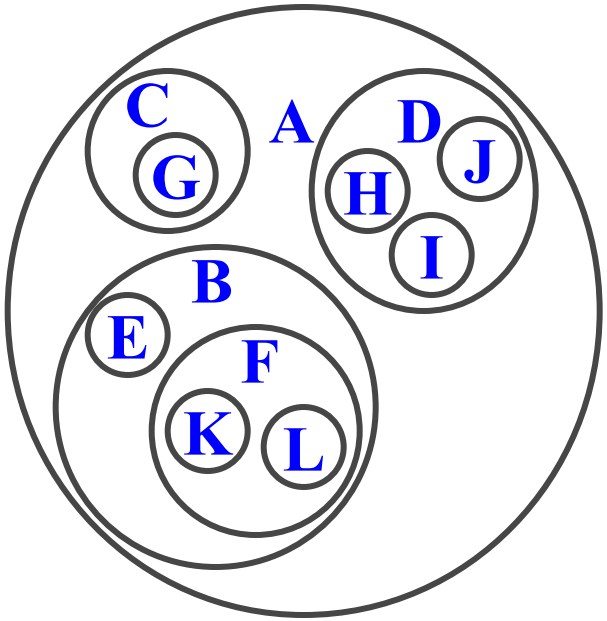
(2) 当n>1时，其余结点可分为m (m>0)个互不相交的有限集$T_1, T_2, \ldots, T_m$，其中每一棵子集本身又是一棵符合本定义的树，称为根root的子树。

树的表示



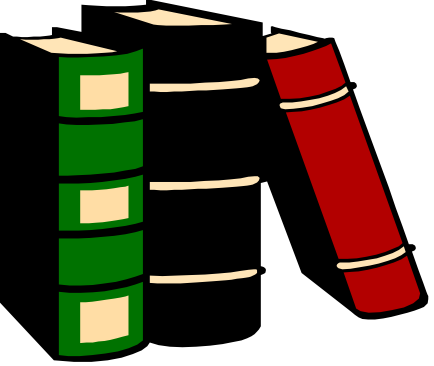
- 树的其他表示

嵌套集合、广义表表示、凹入表示



(A(B(E, F(K,L)), C(G), D(H, I, J)))

```
A**********
  B**********
    E**********
    F**********
      K**********
      L**********
  C**********
    G**********
  D**********
    H**********
    I**********
    J**********
```



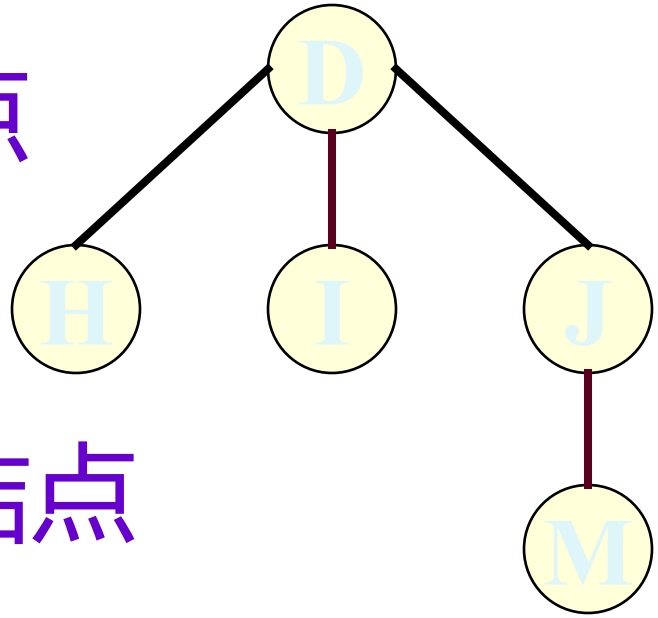
基 本 术 语

结点： 数据元素+若干指向子树的分支

结点的度： 分支的个数

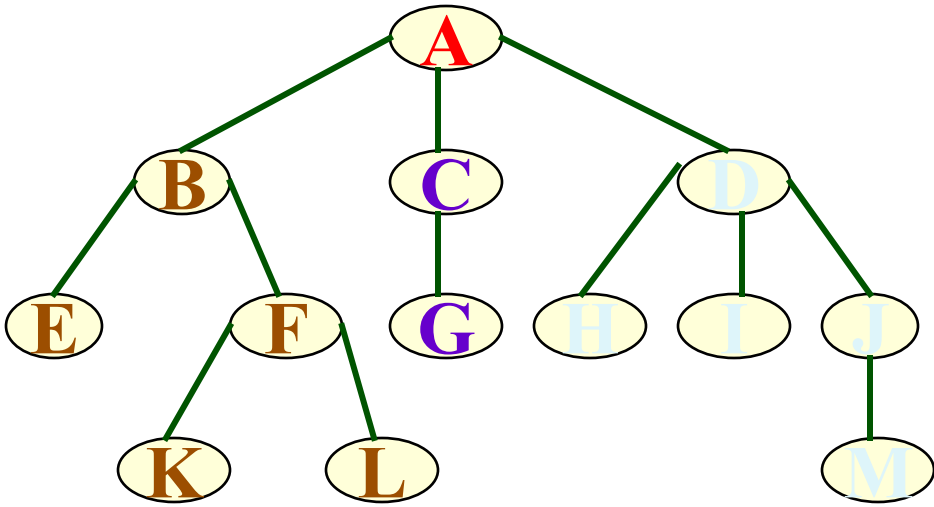
树的度： 树中所有结点的度的最大值

叶子结点： 度为零的结点



分支结点： 度大于零的结点

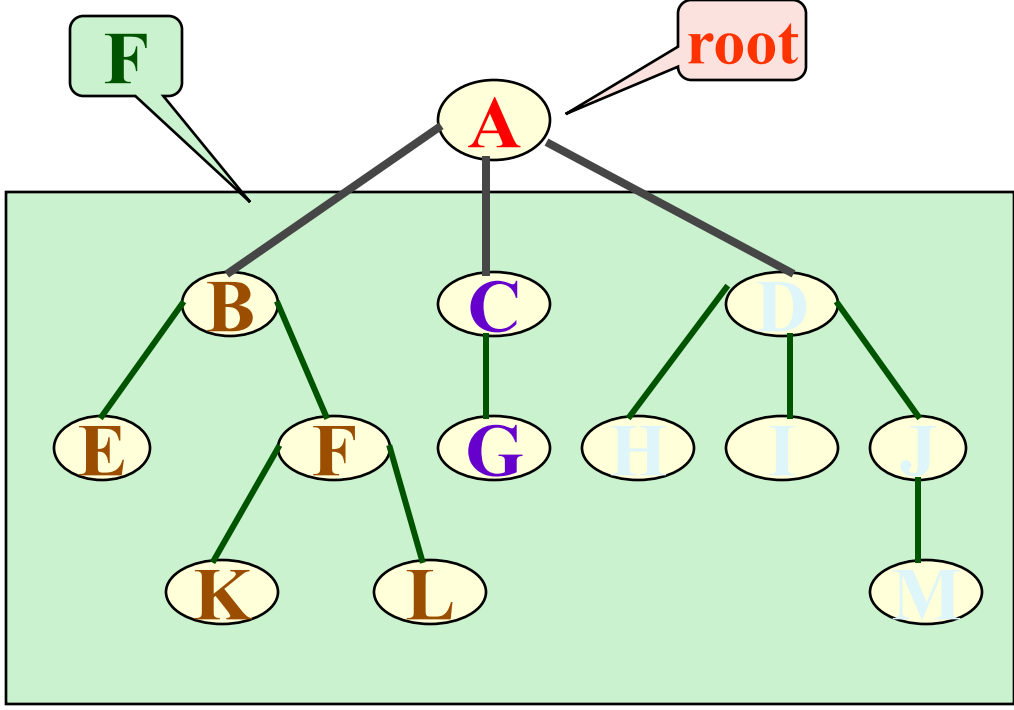
(从根到结点的)**路径：**
由从**根**到该结点所经分支和结点构成



孩子结点、**双亲**结点
兄弟结点、**堂兄弟**结点
祖先结点、**子孙**结点

结点的层次: 假设根结点的层次为1，第l层的结点的子树根结点的层次为$l+1$

树的深度： 树中叶子结点所在的最大层次



森林：

是m（m≥0）棵互不相交的树的集合

任何一棵非空树是一个二元组

Tree = （root，F）

其中：root 被称为根结点

F 被称为子树森林

有向树：

（1）有确定的根；

（2）树根和子树根之间为有向关系。

有序树：

子树之间存在确定的次序关系。

无序树：

子树之间不存在确定的次序关系。

基本操作：

- 查 找 类
- 插 入 类
- 删 除 类

查找类：

Root(T) // 求树的根结点

Value(T, cur_e) // 求当前结点的元素值

Parent(T, cur_e) // 求当前结点的双亲结点

LeftChild(T, cur_e) // 求当前结点的最左孩子

RightSibling(T, cur_e) // 求当前结点的右兄弟

TreeEmpty(T) // 判定树是否为空树

TreeDepth(T) // 求树的深度

TraverseTree(T, Visit()) // 遍历

插入类：

InitTree(&T) // 初始化置空树

CreateTree(&T, definition)
// 按定义构造树

Assign(T, cur_e, value)
// 给当前结点赋值

InsertChild(&T, &p, i, c)
// 将以c为根的树插入为结点p的第i棵子树



删除类：

ClearTree(&T) // 将树清空

DestroyTree(&T) // 销毁树的结构

DeleteChild(&T, &p, i)

// 删除结点p的第i棵子树



对比树型结构和线性结构的结构特点



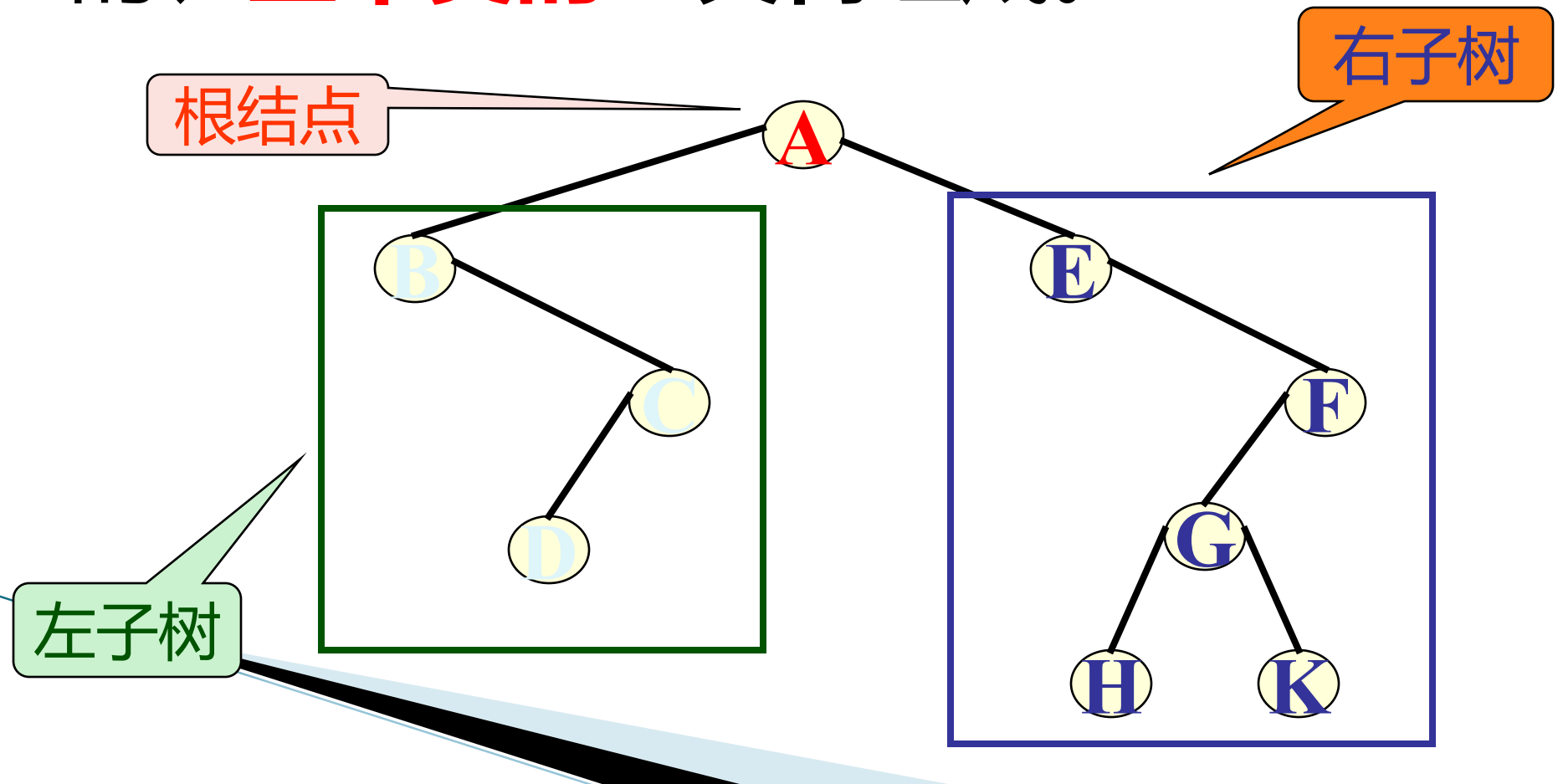
线性结构	树型结构
第一个数据元素 (无前驱)	根结点 (无前驱)
最后一个数据元素 (无后继)	多个叶子结点 (无后继)
其它数据元素 (一个前驱、 一个后继)	其它数据元素 (一个前驱、 多个后继)

6.2

二叉树的类型定义



二叉树或为**空树**，或是由一个**根结点**加上**两棵**分别称为**左子树**和右子树的、**互不交的二叉树**组成。



二叉树的五种基本形态：

空树

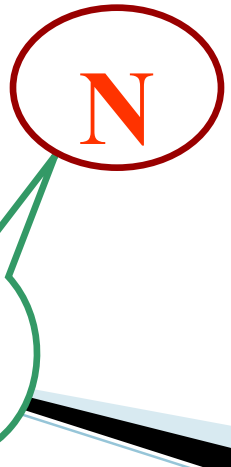


只含根结点

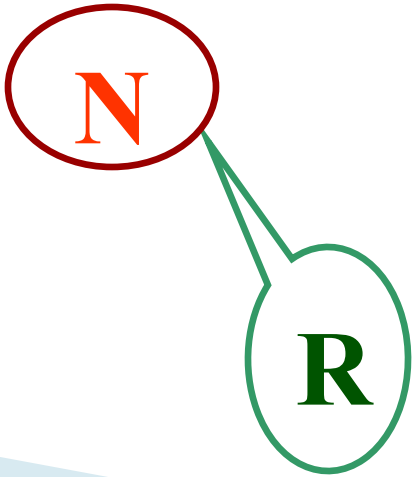


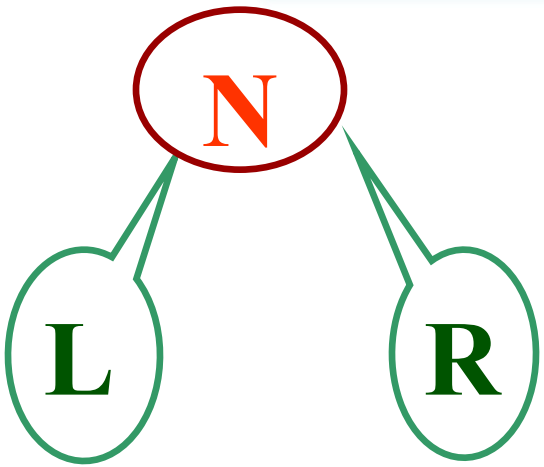
左右子树均不为空树

右子树为空树



左子树为空树





二叉树的主要基本操作：



查 找 类



插 入 类



删 除 类

Root(T); Value(T, e); Parent(T, e);

LeftChild(T, e); RightChild(T, e);

LeftSibling(T, e); RightSibling(T, e);

BiTreeEmpty(T); BiTreeDepth(T);

PreOrderTraverse(T, Visit());

InOrderTraverse(T, Visit());

PostOrderTraverse(T, Visit());

LevelOrderTraverse(T, Visit());



InitBiTree(&T);

Assign(T, &e, value);

CreateBiTree(&T, definition);

InsertChild(T, p, LR, c);



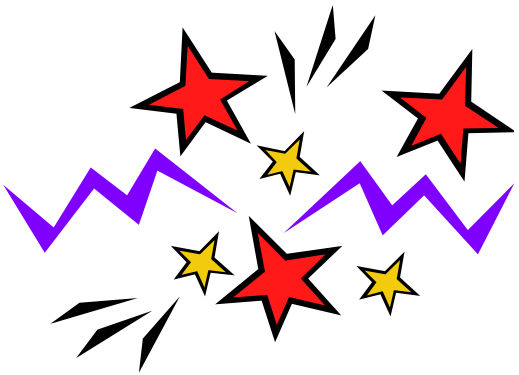
ClearBiTree(&T);

DestroyBiTree(&T);

DeleteChild(T, p, LR);



二叉树的重要特性



性质 1 ：

在二叉树的第 i 层上至多有 2^{i-1} 个结点。($i \geq 1$)

用归纳法证明：

归纳基： $i = 1$ 层时，只有一个根结点：

$$2^{i-1} = 2^0 = 1;$$

归纳假设： 假设对所有的 j，$1 \leq j < i$，命题成立；

归纳证明： 二叉树上每个结点至多有两棵子树，

则第 i 层的结点数 $= 2^{i-2} \times 2 = 2^{i-1}$ 。

推广：深度为h的k叉树中，第i层上最多具有 k^{i-1} 个结点。

性质 2：

- 深度为 k 的二叉树上至多含 2^k-1 个结点（k≥1）。

证明：

基于上一条性质，深度为 k 的二叉树上的结点数至多为

$$2^0+2^1+\cdots\cdots+2^{k-1}=2^k-1$$ 。

推广：深度为h的k叉树至多含 $(k^h-1)/(k-1)$ 个结点。

性质 3：

- 对任何一棵二叉树，若它含有n_0个叶子结点、n_2个度为 2 的结点，则必存在关系式：$n_0 = n_2+1$。

证明：利用结点总数=结点总度数+1

$$n_0+n_1+n_2=n_1+2*n_2+1 \Rightarrow n_0=n_2+1$$

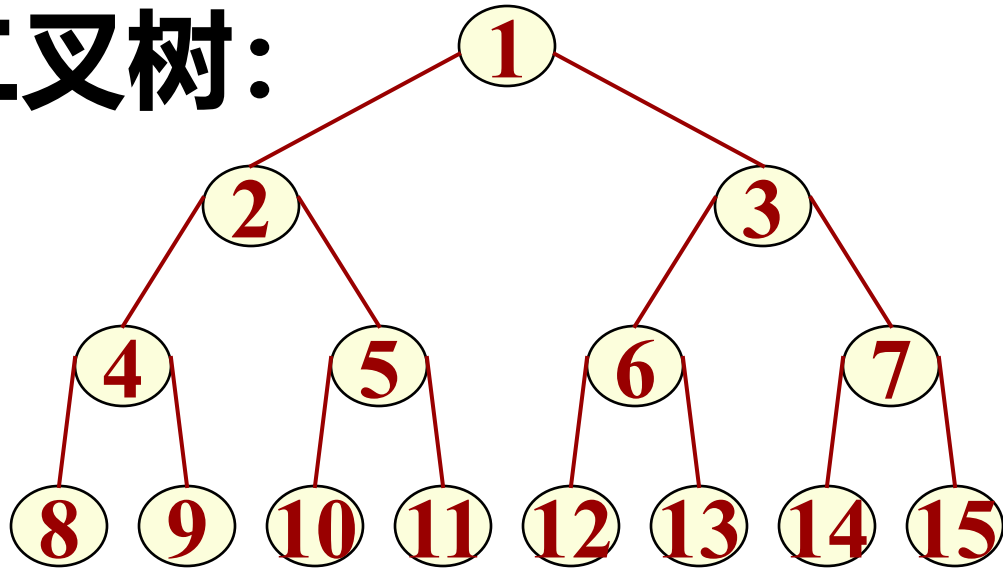
推广：已知一棵树度为m的树中有n_1个度为1的结点，n_2个度为2的结点，…n_m个度为m的结点，问该树中有多少片叶子？

证明：根据结点总数=结点总度数+1

$$n_0+n_1+n_2+\ldots+n_m=n_1+2*n_2+\ldots+m*n_m+1$$

$$\Rightarrow \quad n_0=1+n_2+\ldots+(m-1)n_m$$

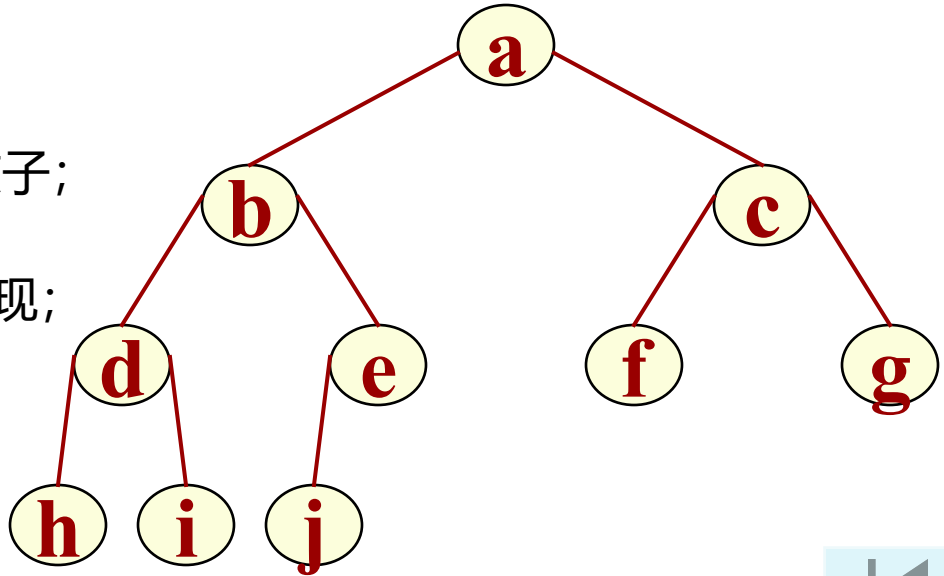
两类特殊的二叉树：



满二叉树：指的是深度为k且含有2^k-1个结点的二叉树。

完全二叉树：树中所含的 n 个结点和满二叉树中**编号为 1 至 n 的结点**一一对应。

1. 前k-1层为满二叉树；
2. 第k层结点依次占据最左边的位置；
3. 一个结点有右孩子，则它必有左孩子；
4. 度为1的结点个数为0或1
5. 叶结点只能在层次最大的两层上出现；

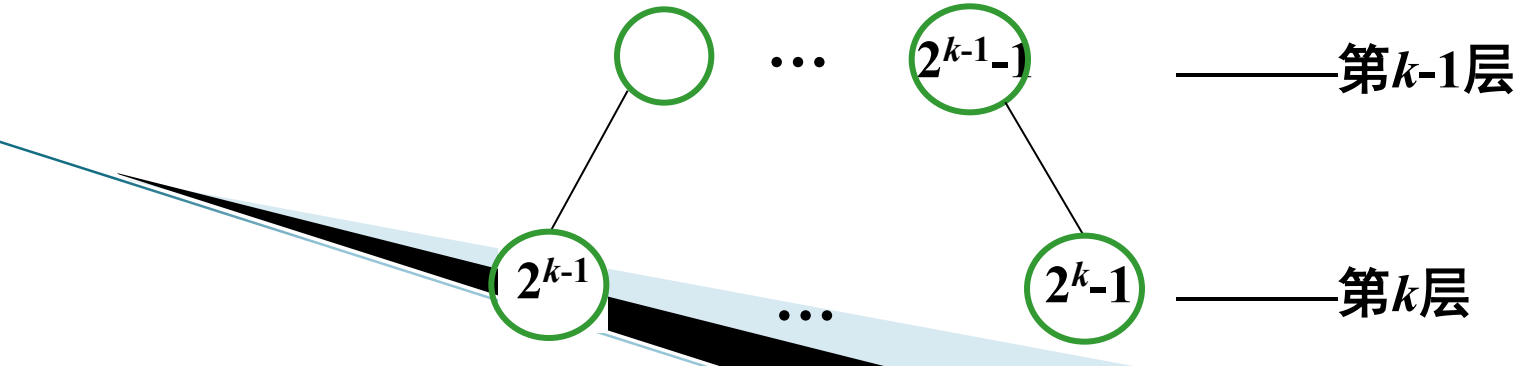


性质 4：

具有 n 个结点的完全二叉树的**深度**为 $\lfloor \log_2 n \rfloor + 1$。

证明：假设具有n个结点的完全二叉树的深度为k，根据完全二叉树的定义和性质2，有下式成立

$$2^{k-1} \leq n < 2^k$$



性质 4：

- 具有 n 个结点的完全二叉树的**深度**为 $\lfloor \log_2 n \rfloor + 1$ 。

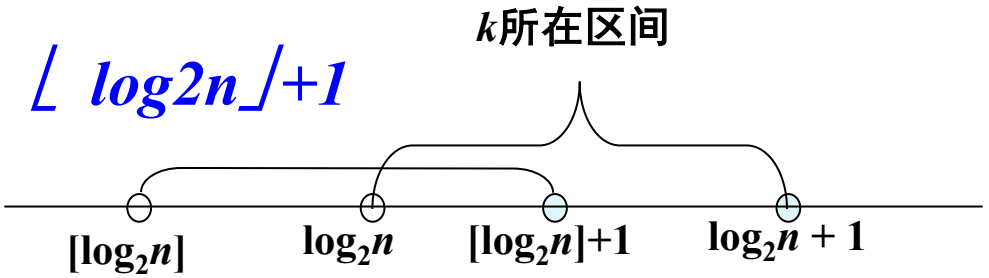
证明：假设具有n个结点的完全二叉树的深度为k，根据完全二叉树的定义和性质2，有下式成立

$$2^{k-1} \le n < 2^k$$

对不等式取对数，有： $k-1 \le \log_2 n < k$

即： $\log_2 n < k \le \log_2 n + 1$

由于k是整数，故必有 $k = \lfloor \log_2 n \rfloor + 1$



性质 5：

若对含 n 个结点的完全二叉树从上到下且从左至右进行 1 至 n 的编号，则对完全二叉树中任意一个编号为 i 的结点：

(1) 若 $i=1$，则该结点是二叉树的根，无双亲，
否则，编号为 $\lfloor i/2 \rfloor$ 的结点为其**双亲**结点；

(2) 若 $2i>n$，则该结点无左孩子，
否则，编号为 $2i$ 的结点为其**左孩子**结点；

(3) 若 $2i+1>n$，则该结点无右孩子结点，
否则，编号为 $2i+1$ 的结点为其**右孩子**结点。

- 例：若一个完全二叉树有1450个结点，则度为1的结点个数为 1 ，度为2的结点个数为 724 ，叶子结点的个数为 725 ，有 725 个结点有左孩子，有 724 个结点有右孩子；该树的高度为 11 。(性质3、性质4以及完全二叉树的特征)

6.3
二叉树的存储结构



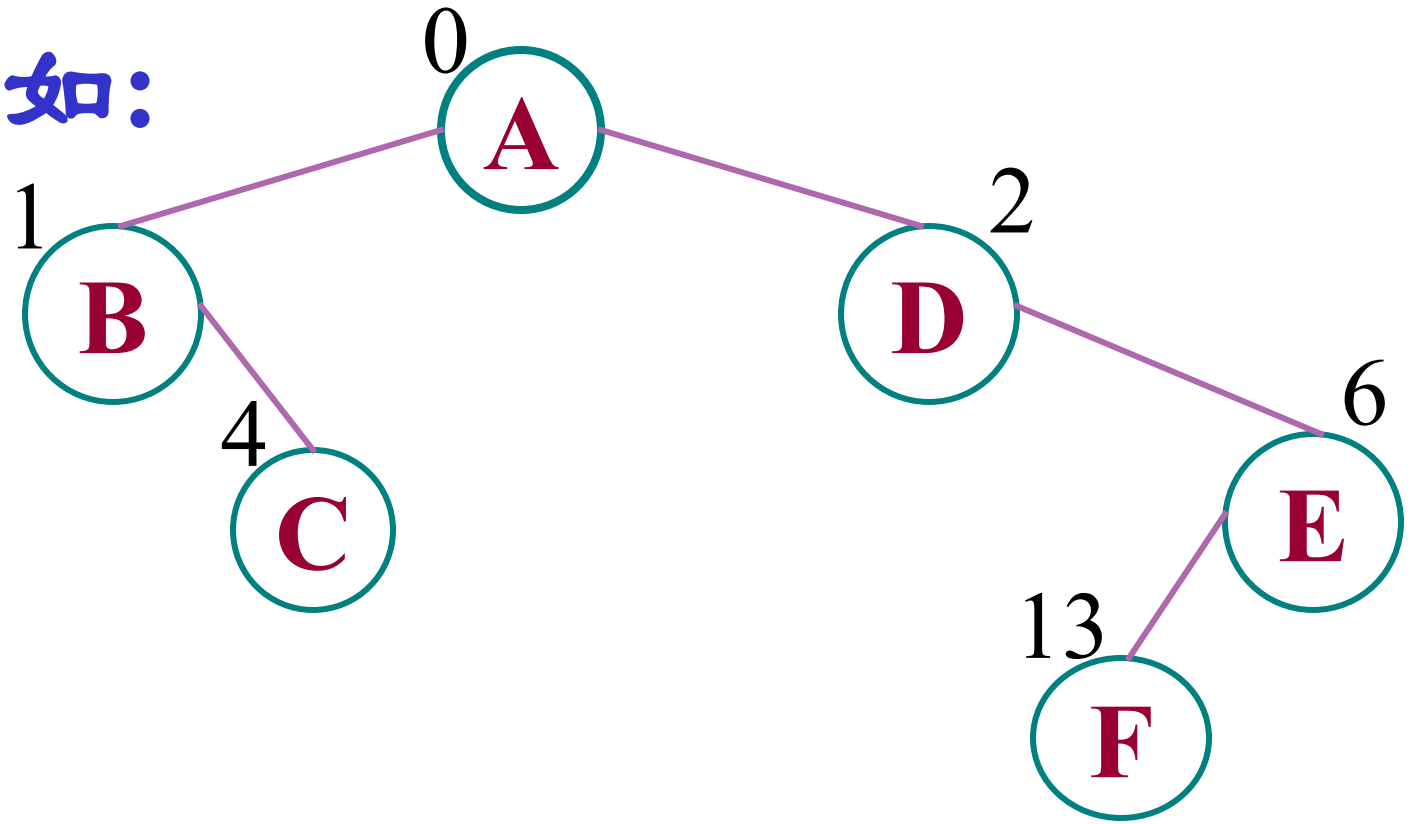
一、 二叉树的顺序存储表示

二、二叉树的链式存储表示

一、二叉树的顺序存储表示

```
#define  MAX_TREE_SIZE  100
            // 二叉树的最大结点数
typedef TElemType  SqBiTree[MAX_
              TREE_SIZE];
            // 0号单元存储根结点
SqBiTree  bt;
```

例如：



0	1	2	3	4	5	6	7	8	9	10	11	12	13
A	B	D		C		E							F

二、二叉树的链式存储表示

1. 二叉链表

2．三叉链表

3．双亲链表

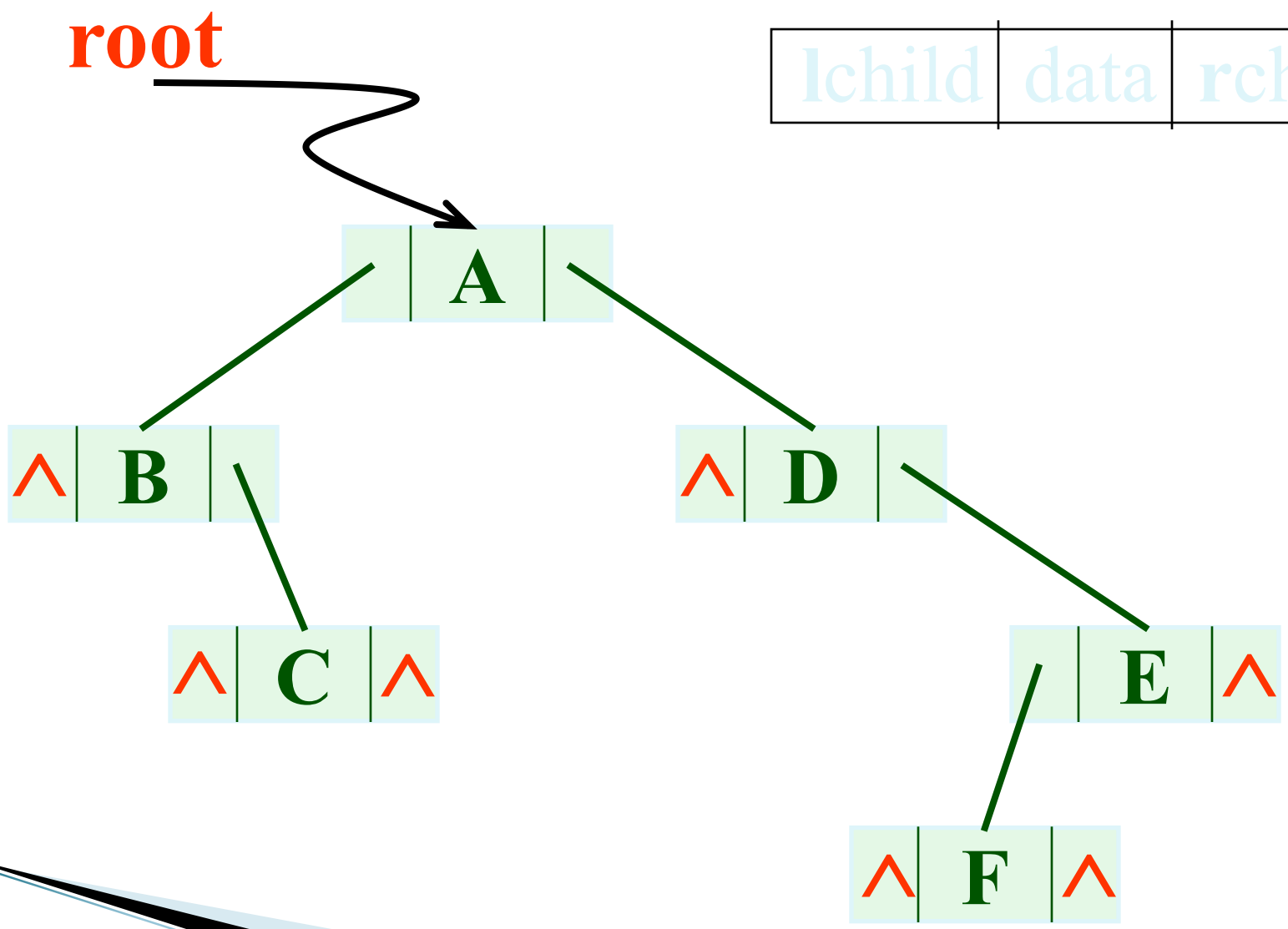
4．线索链表



1. 二叉链表

结点结构:





C 语言的类型描述如下:

```
typedef struct BiTNode { // 结点结构
    TElemType      data;
    struct BiTNode  *lchild, *rchild;
                                  // 左右孩子指针
} BiTNode, *BiTree;
```

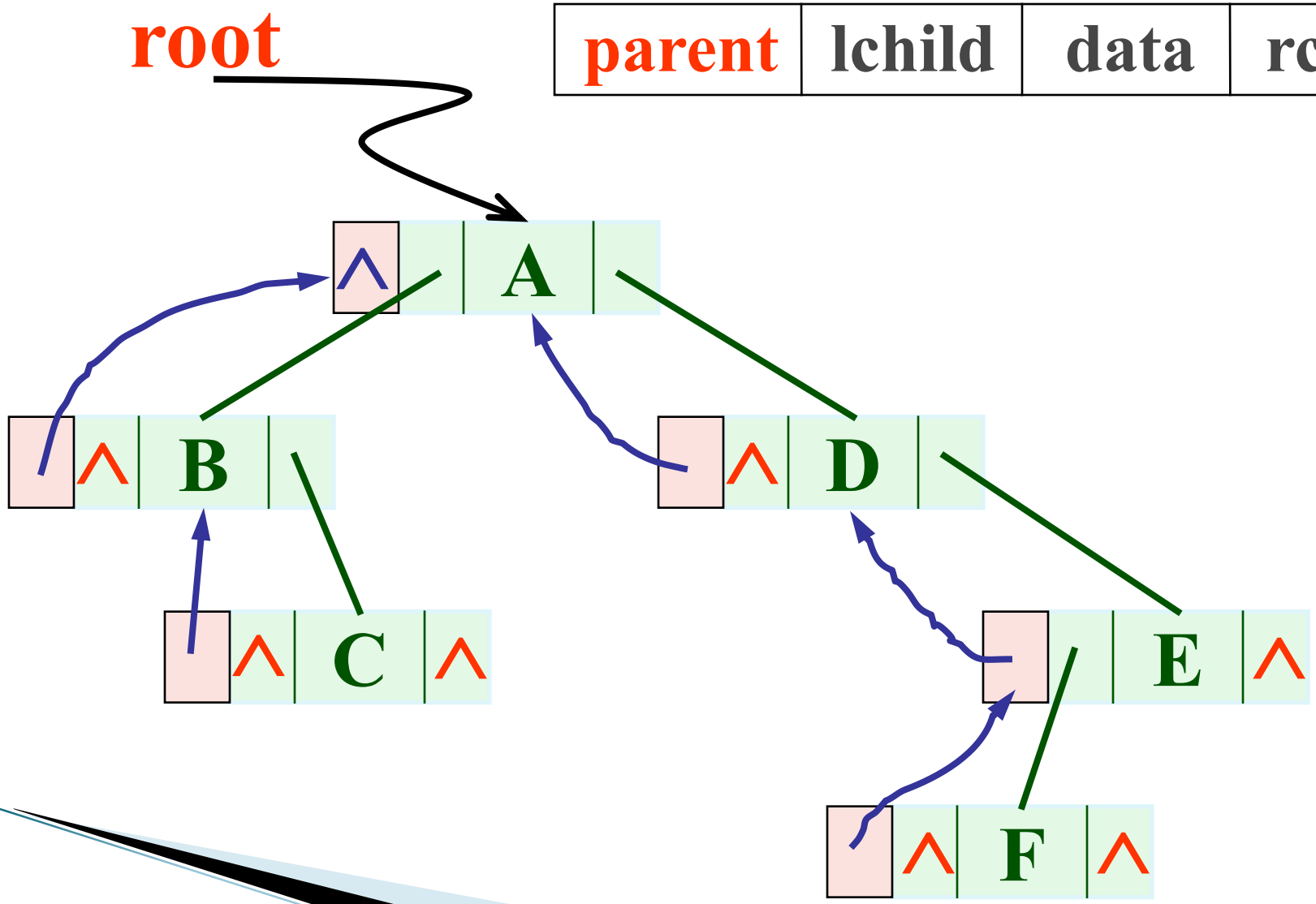
结点结构:



2．三叉链表

结点结构:

parent	lchild	data	rchild



C 语言的类型描述如下:

```
typedef struct TriTNode { // 结点结构
   TElemType       data;
   struct TriTNode  *lchild, *rchild;
                              // 左右孩子指针
   struct TriTNode  *parent;  //双亲指针
} TriTNode, *TriTree;
```

结点结构:

parent	lchild	data	rchild

3．双亲链表

0	B	2	L
1	C	0	R
2	A	-1	
3	D	2	R
4	E	3	R
5	F	4	L
6			

结点结构：

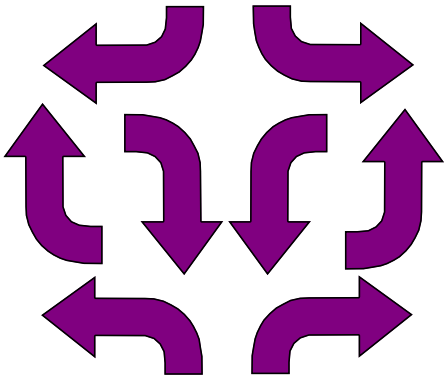


```
typedef struct BPTNode { // 结点结构
    TElemType  data;
    int  *parent;     // 指向双亲的指针
    char  LRTag;    // 左、右孩子标志域
} BPTNode
typedef struct BPTree{ // 树结构
    BPTNode nodes[MAX_TREE_SIZE];
    int  num_node;     // 结点数目
    int  root;                 // 根结点的位置
} BPTree
```



6.4

二叉树的遍历



一、问题的提出

二、先左后右的遍历算法

三、算法的递归描述

四、遍历算法的应用举例

五、遍历算法的非递归描述

一、问题的提出

顺着某一条搜索路径**巡访**二叉树中的结点，使得每个结点**均被访问一次**，而且**仅被访问一次**。

“访问”的含义可以很广，如：输出结点的信息等。

“遍历”是任何类型均有的操作，对线性结构而言，只有一条搜索路径(因为每个结点均只有一个后继)，故不需要另加讨论。而二叉树是非线性结构，**每个结点有两个后继**，则**存在如何遍历**即按什么样的**搜索路径**遍历的问题。

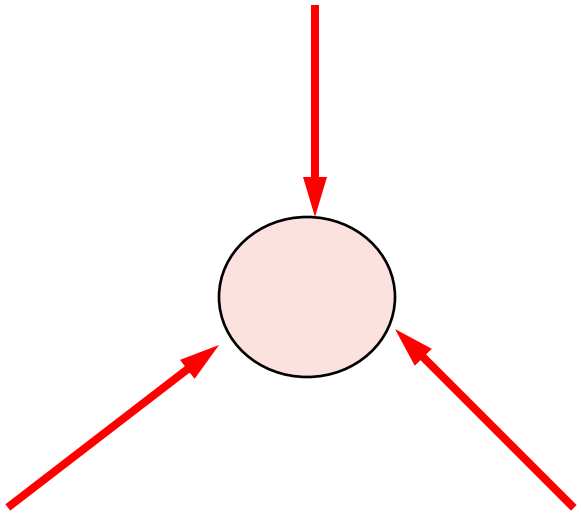
对“二叉树”而言，可以有三条搜索路径：

1．**先上后下**的按层次遍历；

2．**先左**（子树）**后右**（子树）的遍历；

3．**先右**（子树）**后左**（子树）的遍历。

二、先左后右的遍历算法

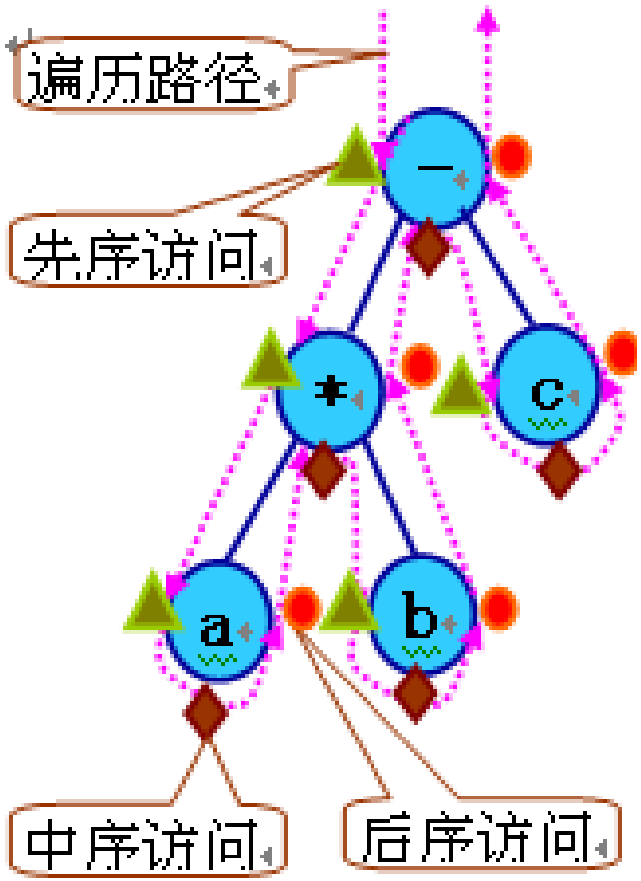


先（根）序的遍历算法

中（根）序的遍历算法

后（根）序的遍历算法

- 先/中/后序遍历的区别
  如右图，三者经过的搜索路线是相同的；只是访问结点的时机不同。每一结点在整个搜索路线中会经过3次：
  - 第一次进入到该结点
    此时访问该结点，称为**先序遍历**
  - 由左子树回溯到该结点
    此时访问该结点，称为**中序遍历**
  - 由右子树回溯到该结点
    此时访问该结点，称为**后序遍历**



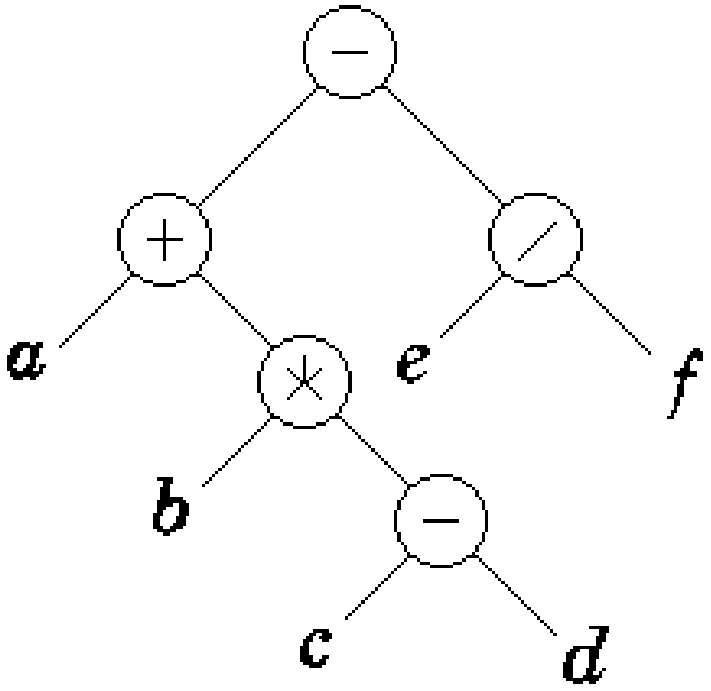
先（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

（1）访问根结点；

（2）先序遍历左子树；

（3）先序遍历右子树。



表达式语法树

遍历结果：

- + a * b - c d / e f

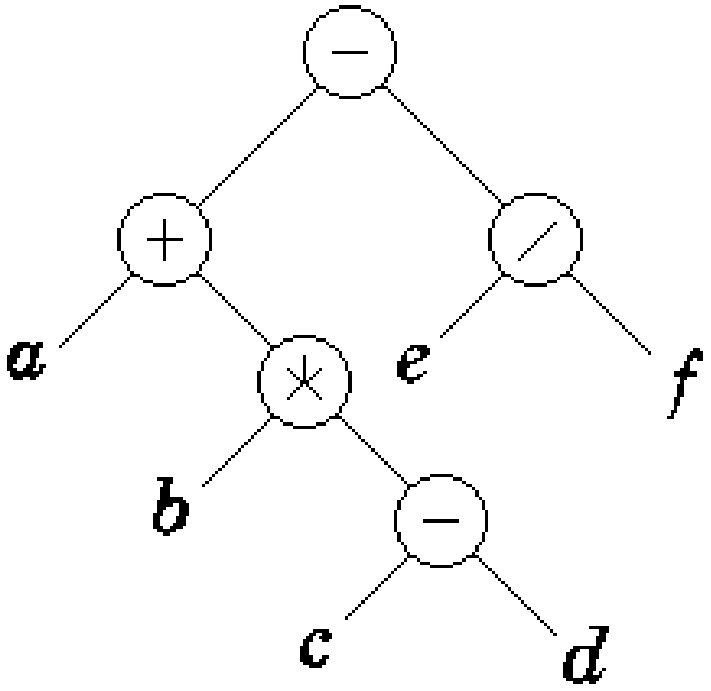
中（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

（1）中序遍历左子树；

（2）访问根结点；

（3）中序遍历右子树。



表达式语法树

遍历结果：

*a + b * c - d - e / f*

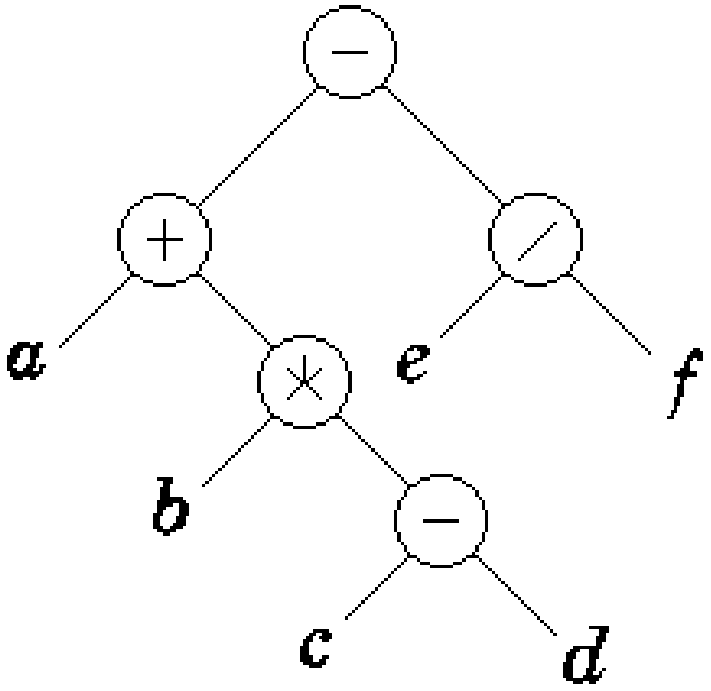
后（根）序的遍历算法：

若二叉树为空树，则空操作；否则，

（1）后序遍历左子树；

（2）后序遍历右子树；

（3）访问根结点。



遍历结果：

*a b c d - * + e f / -*

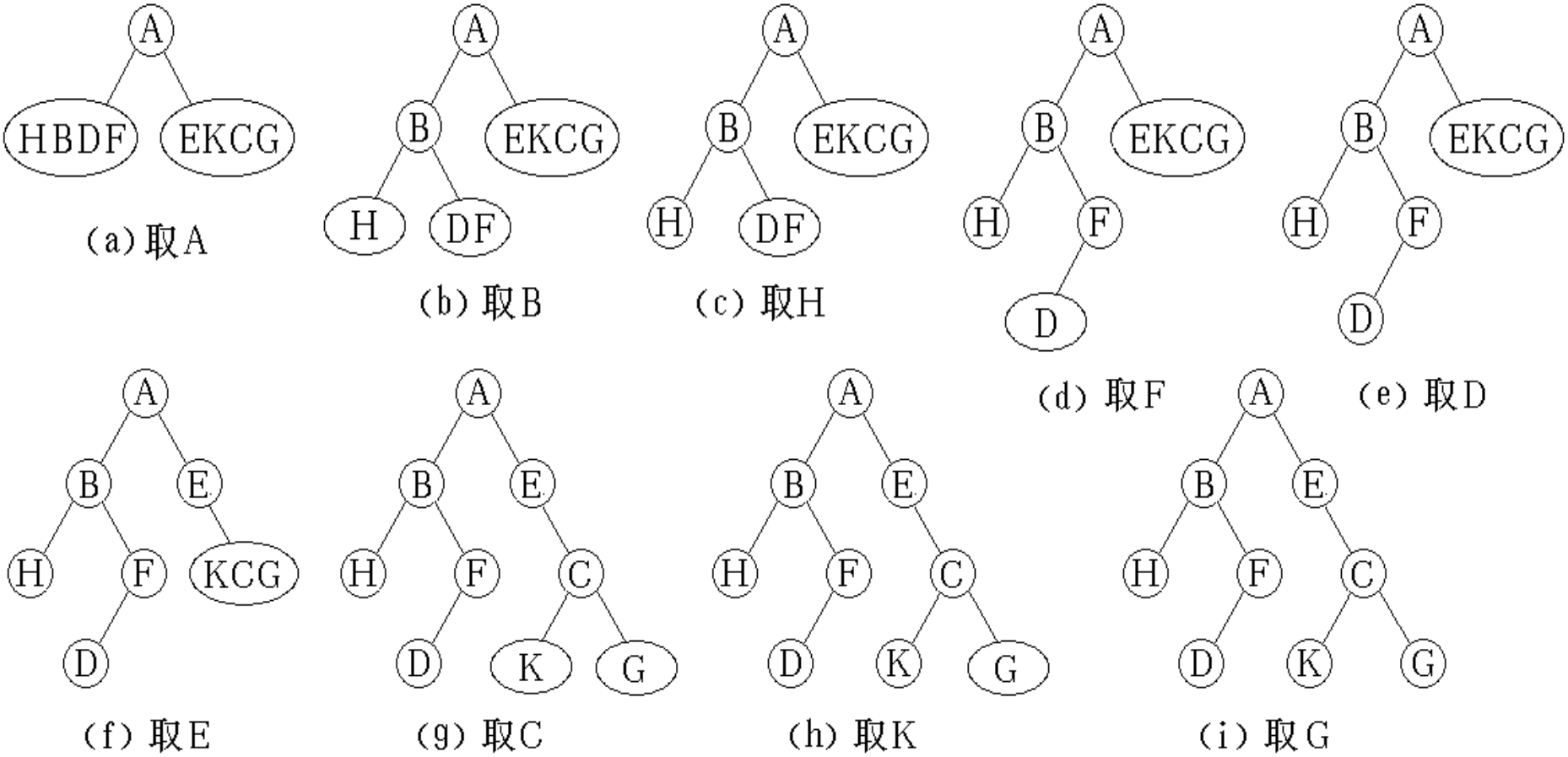
表达式语法树

两个推论

- 若已知一棵二叉树的前序序列和中序序列,则可以唯一的确定这棵二叉树.
- 若已知一棵二叉树的后序序列和中序序列,则也可以唯一的确定这棵二叉树.

例：已知前序序列为{ ABHFDECKG }，中序序列为{ HBDFAEKCG }, 试构造二叉树。

解：过程如下：



算法的递归实现

- 遍历算法的递归实现
  - 二叉树的递归定义性质，决定了它的很多算法都可用递归实现，遍历算法就是其中之一。
  - 对于二叉树的遍历，可以不去具体考虑各子问题(左子树、根、右子树)的遍历结果是什么，而去考虑如何由各子问题的求解结果构成原问题(二叉树)的遍历结果——**递归规律的确定**。必须注意的是，当二叉树小到一定程度，即空树时，应直接给出解答——**递归结束条件及处理**。

三、算法的递归描述

```
void Preorder (BiTree T,
               void( *visit)(TElemType& e))
{  // 先序遍历二叉树
   if (T) {
      visit(T->data);              // 访问结点
      Preorder(T->lchild, visit); // 遍历左子树
      Preorder(T->rchild, visit);// 遍历右子树
   }
}
```



四、遍历算法的非递归描述

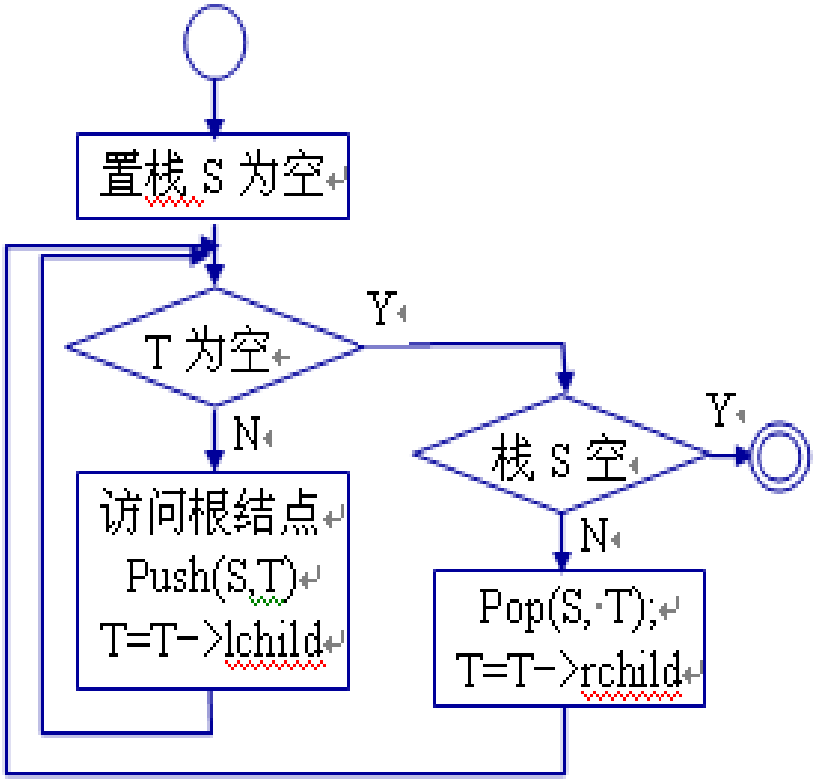
先序遍历的非递归算法

- 思路

假设：T指向要遍历的二叉树的根，若T != NULL 对于非递归算法，引入栈模拟递归工作栈，初始栈为空。

问题：如何用栈来保存信息，使得在先序遍历过左子树后，能利用栈顶信息获取T的右子树的根指针?

方法：访问T->data后，将T入栈，遍历左子树；遍历完左子树返回时，栈顶元素应为T，出栈，再先序遍历T的右子树。



先序遍历的非递归算法

```
void Preorder_I(BiTree T, void (*visit)(TelemType& e))
{  Stack *S;  initStack(S);   t=T;
   while(!(t==NULL&&StackEmpty(S ))
    {   while(t)
          {
           visit(t->data); push(S,t); t=t->lchild;
           }
      if (StackEmpty(S)) return;   // 栈空时结束
       else {
             t = Pop(S);  t=t->rchild;    // 从栈中取出祖先，再转向其右子树
            }
    } // while
}// Preorder_I
```

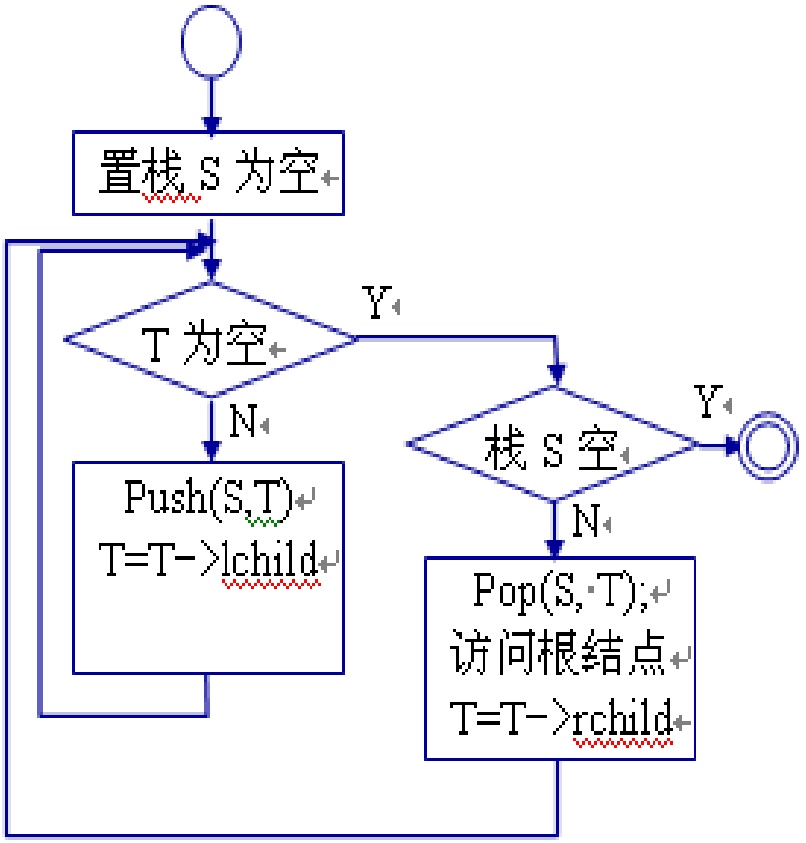
中序遍历的非递归算法

- 思路

T是指向要遍历的二叉树的根，中序遍历要求在遍历完左子树后，访问根，再遍历右子树。

问题：如何用栈来保存信息，使得在中序遍历过左子树后，能利用栈顶信息获取T指针?

方法：先将T入栈，遍历左子树；遍历完左子树返回时，栈顶元素应为T，出栈，访问T->data，再中序遍历T的右子树。



中序遍历算法的非递归描述

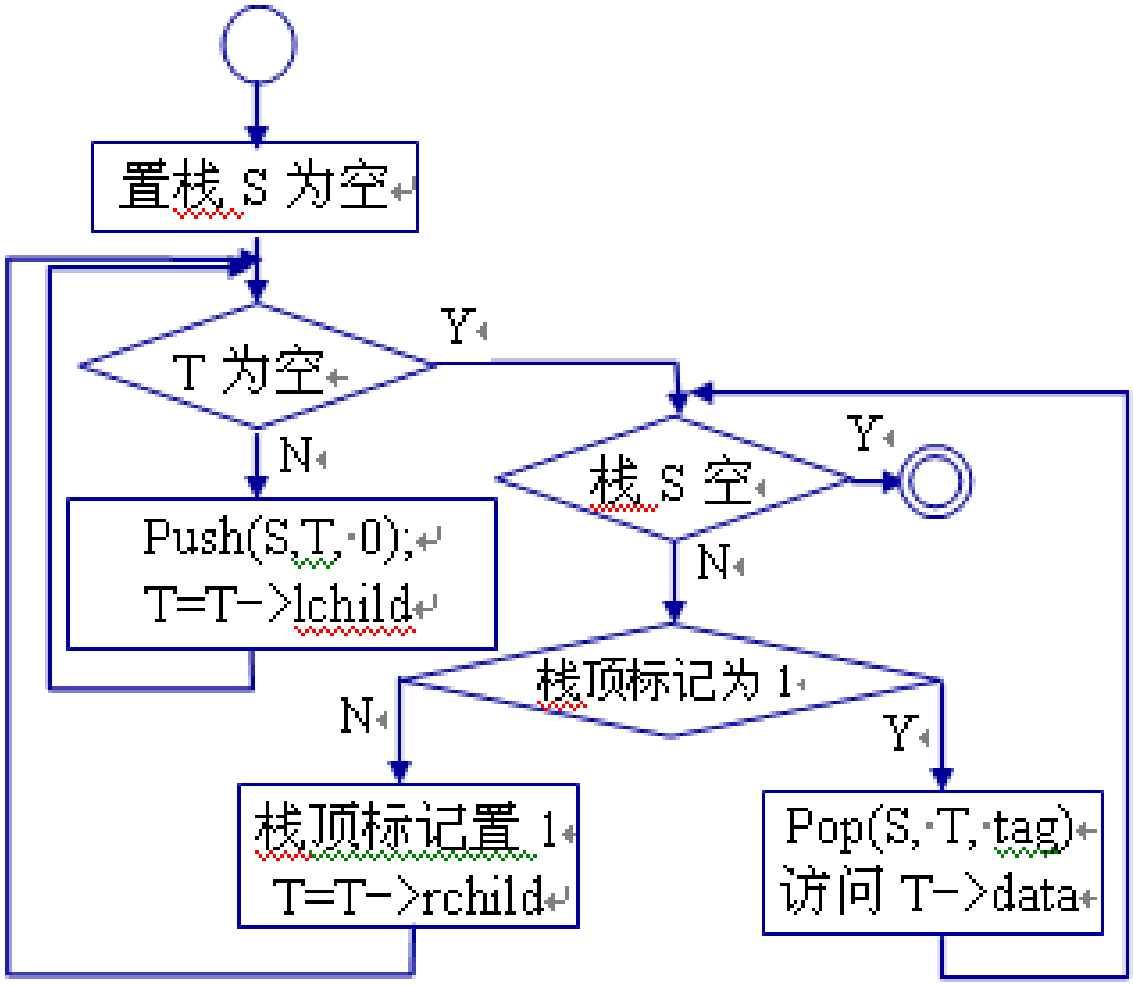
```
void Inorder_I(BiTree T, void (*visit)(TelemType& e))
{  Stack *S;   t=T;
   while(!(t==NULL&&StackEmpty(S ))
    {   while(t)
          { push(S,t);    t=t->lchild; // 先保存根，处理完左子树再访问
           }
      if (StackEmpty(S)) return;   // 栈空时结束
       else {
             t = Pop(S);   visit(t->data); t=t->rchild;
           }
   } // while
}// Inorder_I
```

后序遍历的非递归算法

- 思路
  - T指向要遍历的二叉树的根，后序遍历要求在遍历完左右子树后，再访问根。需要判断根结点的左右子树是否均遍历过。
  - 可采用标记法，结点入栈时，配一个标志tag一同入栈(0：遍历左子树前的现场保护，1：遍历右子树前的现场保护)。
  - 首先将T和tag(为0)入栈，遍历左子树；返回后，修改栈顶tag为1，遍历右子树；最后访问根结点。

```
typedef struct stackElement{
    Bitree          link;
    int     tag;
}stackElemType;
```

后序遍历的非递归算法


置栈 S 为空
T 为空
Y
N
Push(S,T,0);
T=T->lchild
栈 S 空
Y
N
栈顶标记为 1
N
Y
栈顶标记置 1
T=T->rchild
Pop(S,T,tag)
访问 T->data

后序遍历的非递归算法

```
void Postorder_I(BiTree T, void (*visit)(TelemType& e))
{  Stack *S;   t=T;
   while(!(t==NULL&&StackEmpty(S ))
     { if(t){ push(S,(t,1)); t=t->lchild;}
       else{ t=top(s). link;  sign=top(s).tag; pop(s);
             if(sign==1){push(S,(t,2);t=t->rchild;)
             else{Visite(t->data); t=NULL; }
 } // while
}// Postorder_I
```

```
typedef struct stackElement{
    Bitree          link;
    int     tag;
}stackElemType;
```

五、遍历算法的应用举例

1.统计二叉树中叶子结点的个数(先序遍历)

2.求二叉树的深度(后序遍历)

3.复制二叉树(后序遍历)

4.建立二叉树的存储结构

5.删除并释放二叉树中以元素值为x的结点作为根的各子树

6.求位于二叉树先序序列中第k个位置的结点的值

分析问题本身的特征，选择合适的遍历次序！

应用递归编写算法，简洁！

1、统计二叉树中叶子结点的个数

算法基本思想:

先序(或中序或后序)遍历二叉树，在遍历过程中查找叶子结点，并计数。

由此，**需在遍历算法中增添一个“计数”的参数**，并将算法中“访问结点”的操作改为：**若是叶子，则计数器增1。**

```
void CountLeaf (BiTree T,  int& count){
  if ( T ) {
    if ((!T->lchild)&& (!T->rchild))
      count++;     // 对叶子结点计数
    CountLeaf( T->lchild, count);
    CountLeaf( T->rchild, count);
  } // if
} // CountLeaf
```



2、求二叉树的深度(后序遍历)

算法基本思想:

首先分析**二叉树的深度**和它的**左、右子树深度**之间的关系。

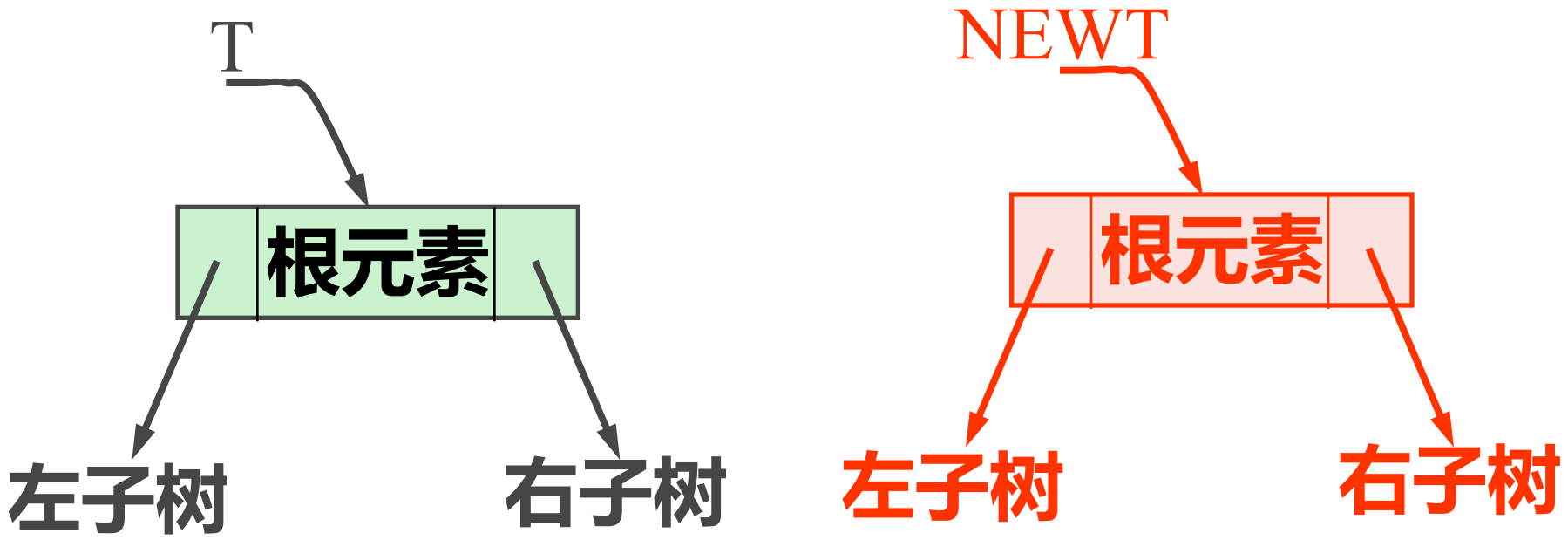
从二叉树深度的定义可知，**二叉树的深度应为其左、右子树深度的最大值加1**。由此，**需先分别求得左、右子树的深度**，算法中“访问结点”的操作为：**求得左、右子树深度的最大值，然后加 1 。**

```
int Depth (BiTree T ){ // 返回二叉树的深度
   if ( !T )    depthval = 0; // 空树深度为 0
   else   {
     depthLeft = Depth( T->lchild );
     depthRight= Depth( T->rchild );
     depthval = 1 + (depthLeft > depthRight  ?
                          depthLeft : depthRight);
   }
   return depthval;
}
```



3、复制二叉树(后序遍历)

其基本操作为：生成一个结点。

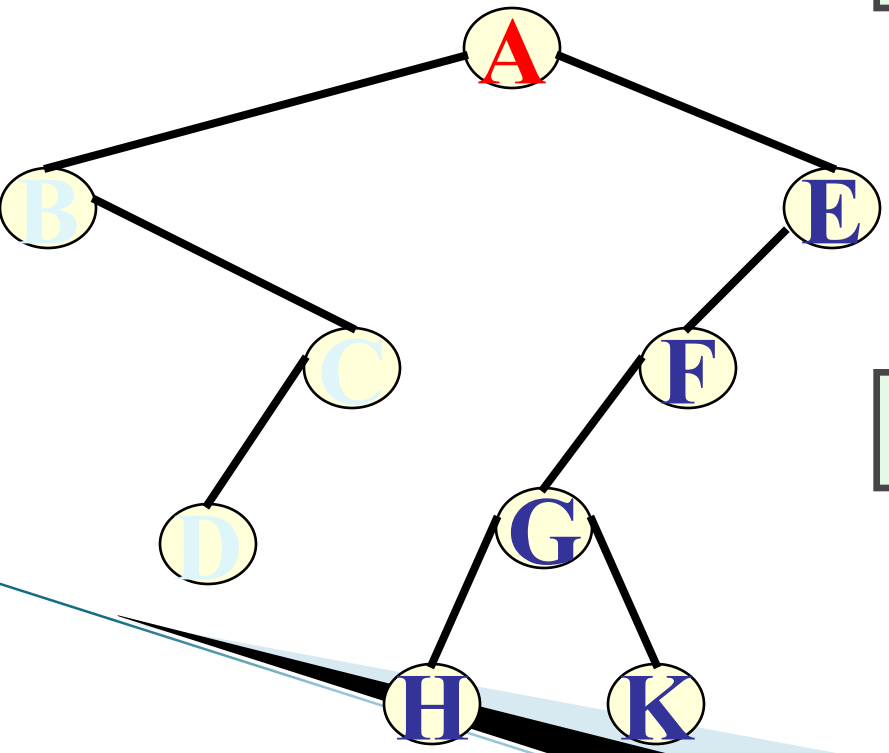


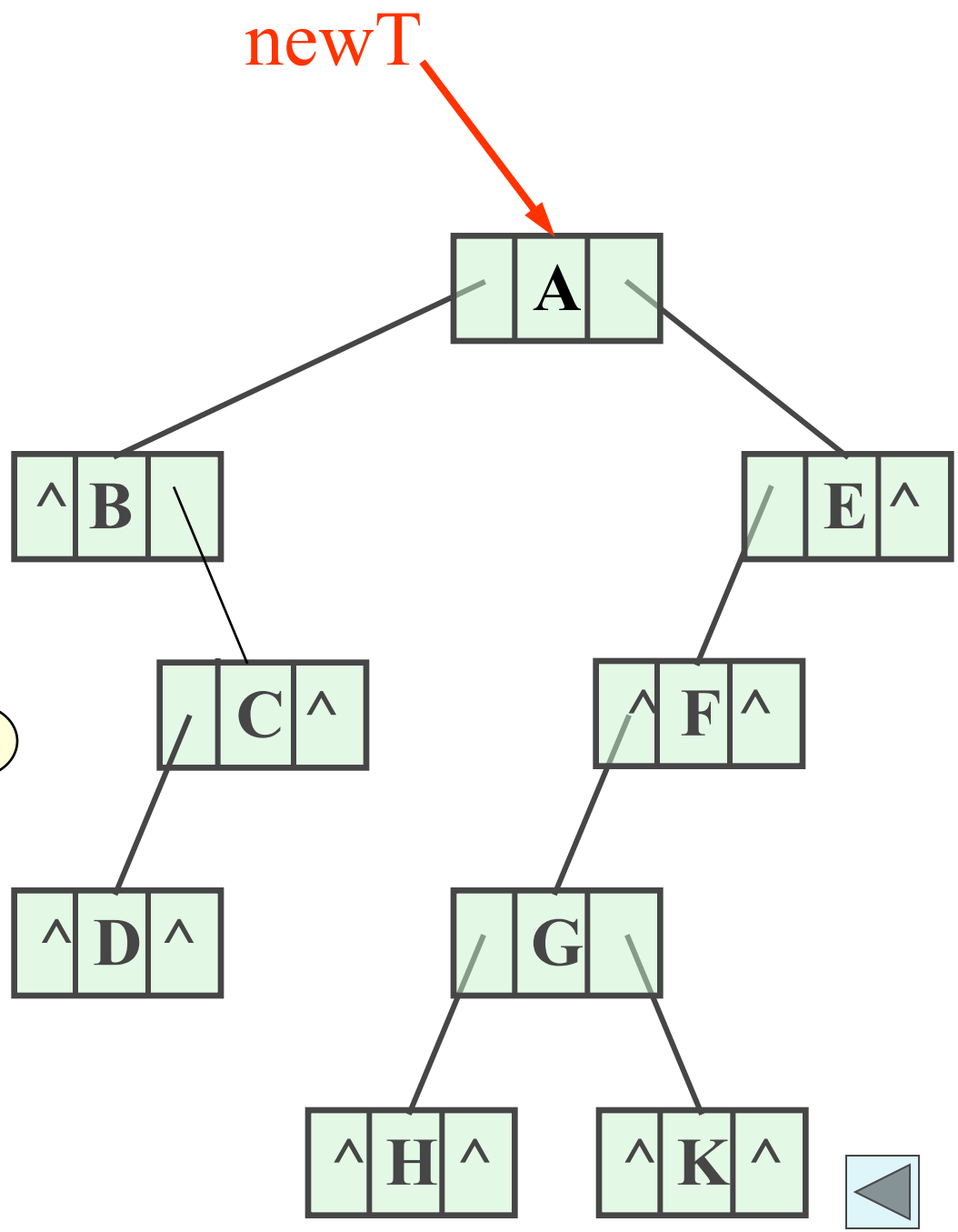
生成一个二叉树的结点
(其数据域为item,左指针域为lptr,右指针域为rptr)

```
BiTNode *GetTreeNode(TElemType item,
        BiTNode *lptr , BiTNode *rptr ){
    if (!(T = (BiTNode*)malloc(sizeof(BiTNode))))
        exit(1);
    T-> data = item;
    T-> lchild = lptr;    T-> rchild = rptr;
    return T;
}
```

```
BiTNode *CopyTree(BiTNode *T) {
    if (!T )    return NULL;
    if (T->lchild )
        newlptr = CopyTree(T->lchild);//复制左子树
    else  newlptr = NULL;
    if (T->rchild )
        newrptr = CopyTree(T->rchild);//复制右子树
    else  newrptr = NULL;
    newT = GetTreeNode(T->data, newlptr, newrptr);
    return newT;
} // CopyTree
```

例如：下列二叉树的复制过程如下：





4、建立二叉树的存储结构

不同的定义方法相应有不同的存储结构的建立算法

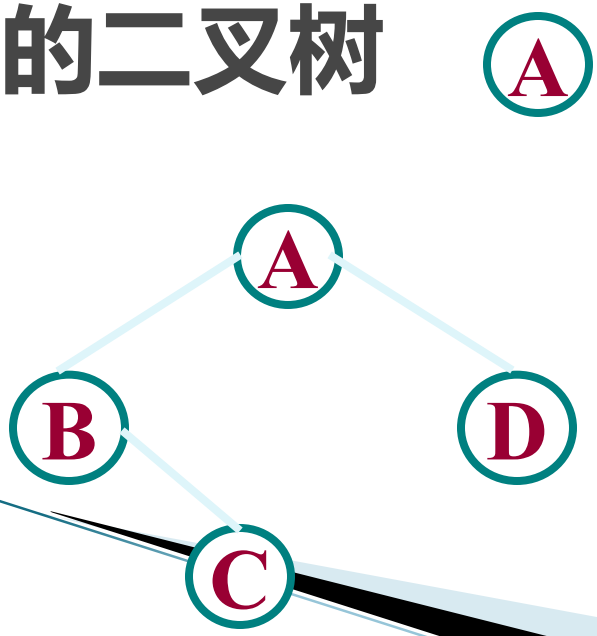


- 以字符串的形式 根 左子树 右子树 定义一棵二叉树

例如:

空树　　以空白字符“ ”表示

只含一个根结点的二叉树　　以字符串“A ”表示



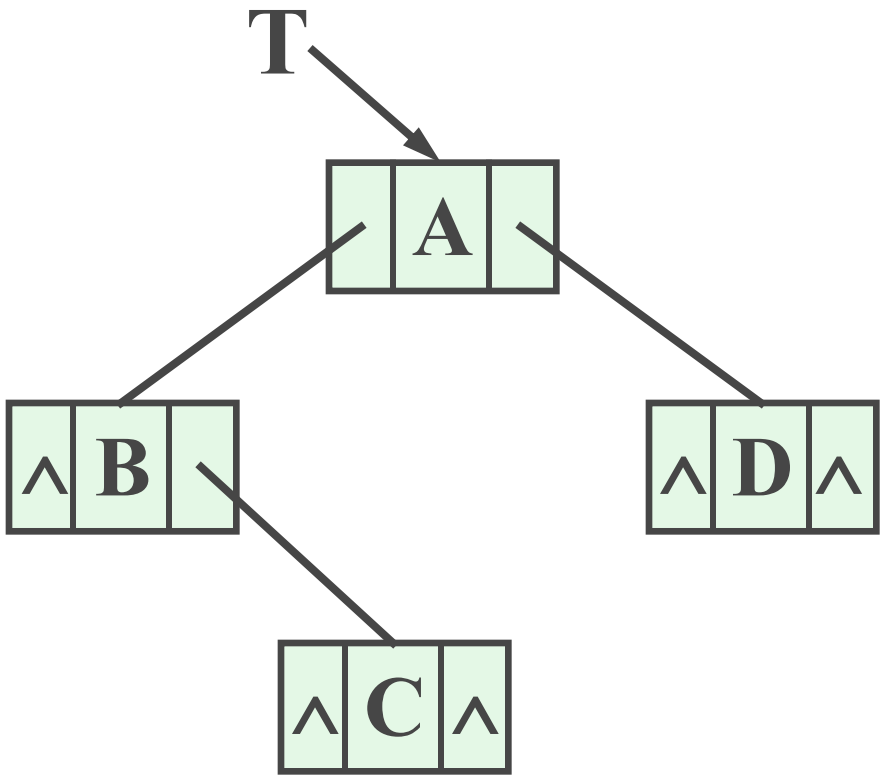
以下列字符串表示

A(B(,C(,)),D(,))

```
Status CreateBiTree(BiTree &T) {
    scanf(&ch);
    if (ch==' ') T = NULL; // 空格是空子树标记
    else {
      if (!(T = (BiTNode *)malloc(sizeof(BiTNode))))
        exit(OVERFLOW);
      T->data = ch;              // 生成根结点
      CreateBiTree(T->lchild);   // 构造左子树
      CreateBiTree(T->rchild);   // 构造右子树
    }
    return OK; } // CreateBiTree
```

上页算法执行过程举例如下：

A B ■ C ■ ■ D ■ ■



按给定的表达式建相应二叉树

- **由先缀表示式建树**

例如：已知表达式的先缀表示式

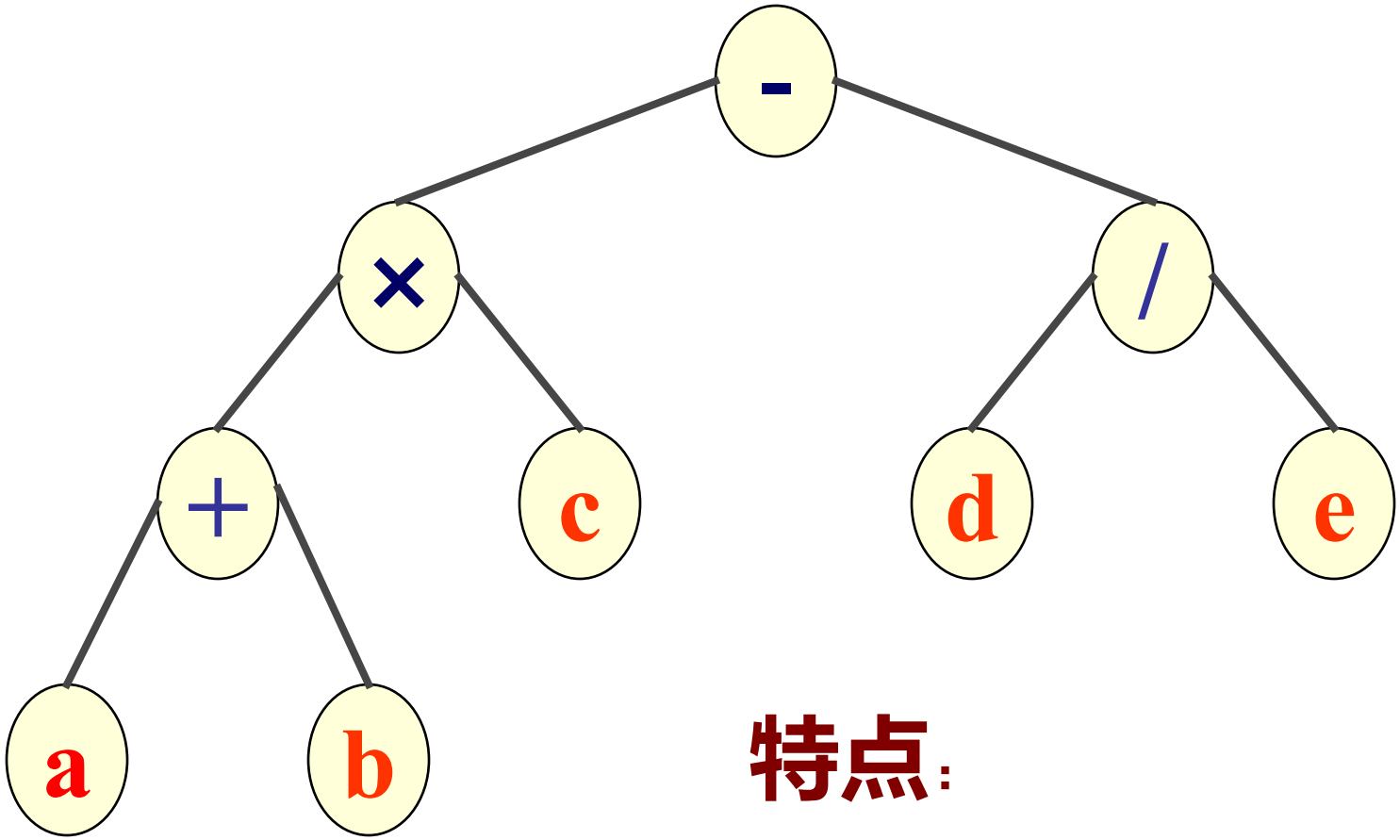
- × + a b c / d e

- **由原表达式建树**

例如：已知表达式 **(a+b)×c – d/e**



对应先缀表达式 -×+ a b c / d e的二叉树



特点：

操作数为**叶子**结点

运算符为**分支**结点

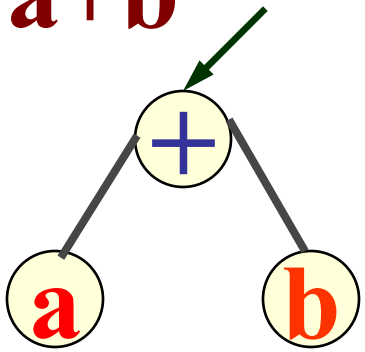
由先缀表示式建树的算法的基本操作：

```
scanf(&ch);
if ( In(ch, 字母集 ))   建叶子结点;
else  { 建根结点;
        递归建左子树;
        递归建右子树;
}
```

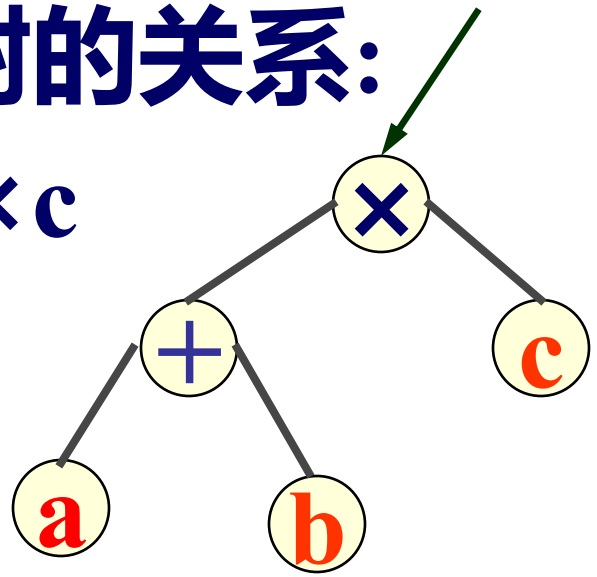


分析表达式和二叉树的关系:

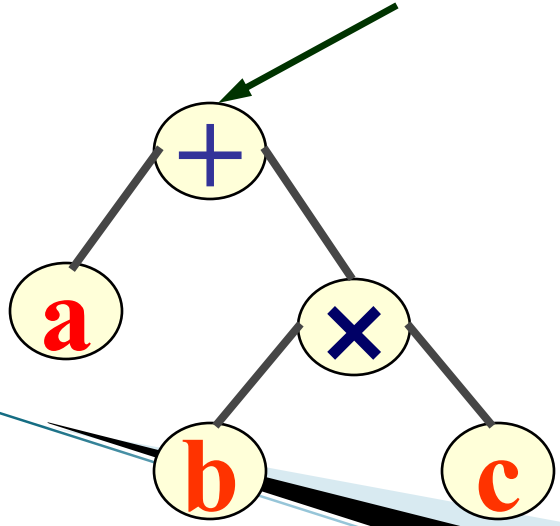
a+b



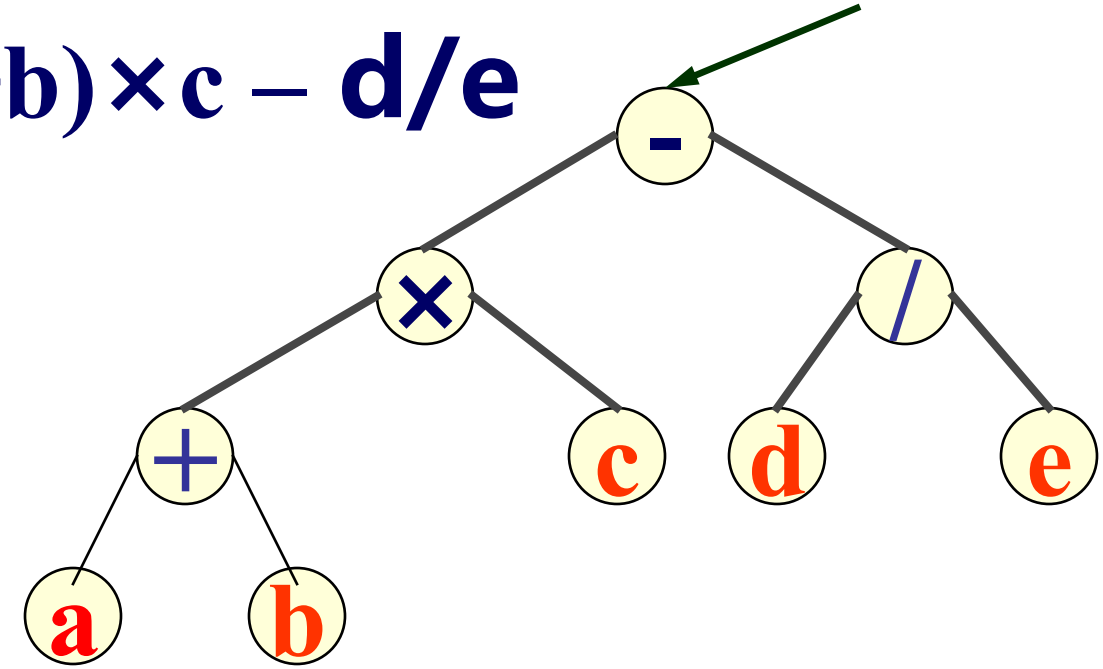
(a+b)×c



a+b×c



(a+b)×c – d/e



基本操作:

```
scanf(&ch);
if (In(ch, 字母集 ))  { 建叶子结点; 暂存; }
else  if  (In(ch, 运算符集))
   { 和前一个运算符比较优先数;
     若当前的优先数“高”，则暂存;
     否则建子树;
   }
```

```
void CrtExptree(BiTree &T, char exp[] ) {
  InitStack(S);  Push(S, '#');  InitStack(PTR);
  p = exp;  ch = *p;
  while (!(GetTop(S)=='#' && ch=='#')) {
    if (!IN(ch, OP))   CrtNode( t, ch );
                                  // 建叶子结点并入栈
                … …
    else {              }
    if ( ch!= '#' ) { p++;  ch = *p;}
  } // while
  Pop(PTR, T);
} // CrtExptree
```



```
switch (ch) {
    case '(' : Push(S, ch); break;
    case ')' : Pop(S, c);
               while (c!= '(' ) {
                  CrtSubtree( t, c);  // 建二叉树并入栈
                  Pop(S, c)      }
               break;
    defult :

} // switch
```



```
while(!Gettop(S, c) && ( precede(c,ch)))
{
    CrtSubtree( t, c);
    Pop(S, c);
}
if ( ch!= '#' ) Push( S, ch);
break;
```



建叶子结点的算法为：

```
void CrtNode(BiTree& T,char ch)
{
   T=(BiTNode*)malloc(sizeof(BiTNode));
   T->data = char;
   T->lchild = T->rchild = NULL;
   Push( PTR, T );
}
```

建子树的算法为：

```
void CrtSubtree (Bitree& T, char c)
{
   T=(BiTNode*)malloc(sizeof(BiTNode));
   T->data = c;
   Pop(PTR, rc);  T->rchild = rc;
   Pop(PTR, lc);  T->lchild = lc;
   Push(PTR, T);
}
```



由二叉树的先序和中序序列建树

仅知二叉树的先序序列“**abcdefg**”不能唯一确定一棵二叉树，

如果同时已知二叉树的中序序列“**cbdaegf**”，则会如何？

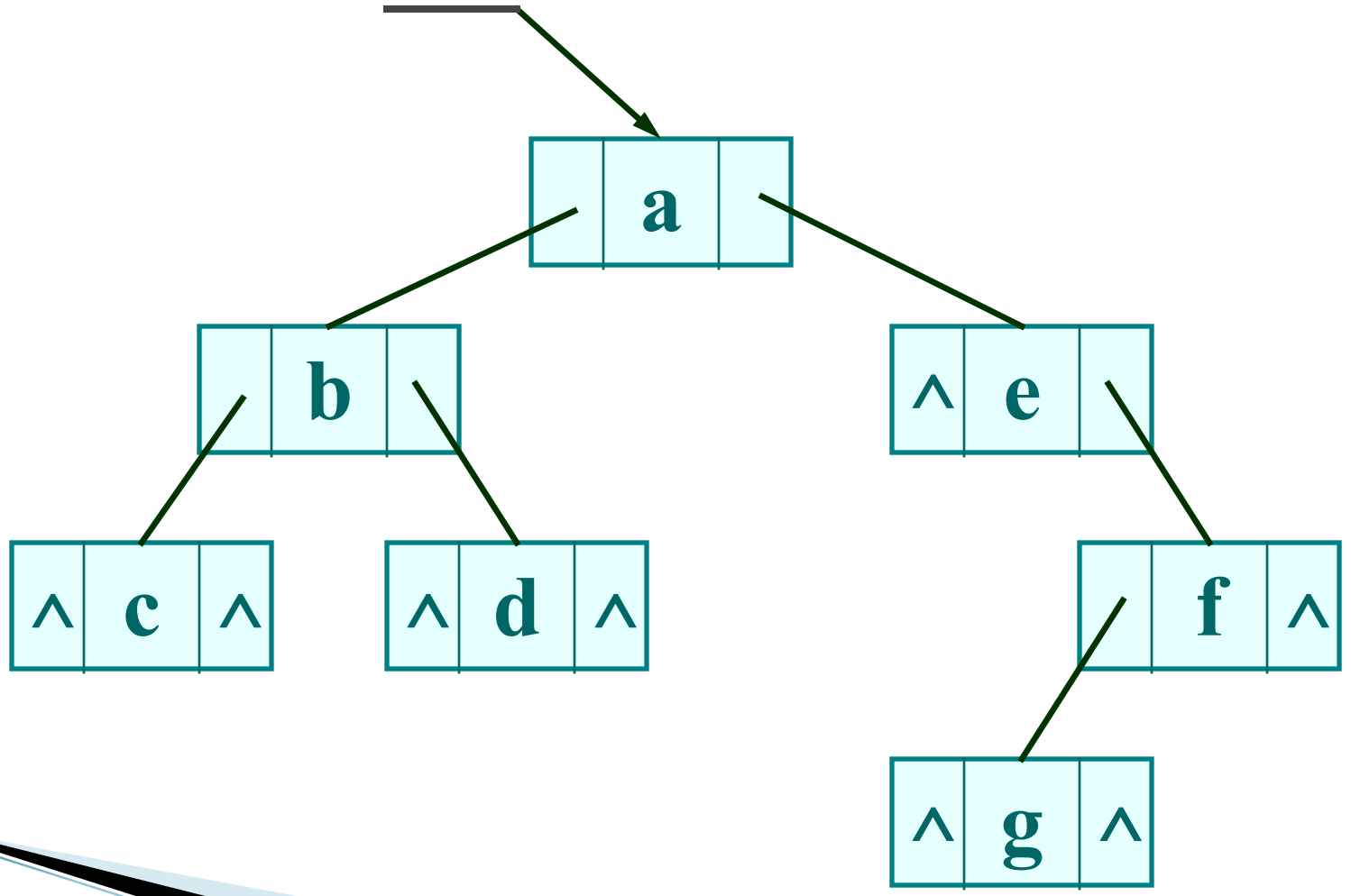
二叉树的先序序列 根 | 左子树 | 右子树

二叉树的中序序列 左子树 | 根 | 右子树

例如:

a b c d e f g 先序序列

c b d a e g f 中序序列



```
void CrtBT(BiTree& T, char pre[], char ino[],
                                    int ps, int is, int n ) {
  // 已知pre[ps..ps+n-1]为二叉树的先序序列，
  // ins[is..is+n-1]为二叉树的中序序列，本算
  // 法由此两个序列构造二叉链表
   if (n==0) T=NULL; // 子序列为空时结束递归
   else {
      k=Search(ino, pre[ps]); // 在中序序列中查询
      if (k== -1)  T=NULL;
      else {                    }
   } //
} // CrtBT
```

```
T=(BiTNode*)malloc(sizeof(BiTNode));
T->data = pre[ps];
if (k==is)  T->Lchild = NULL;
else  CrtBT(T->Lchild, pre[], ino[],
                                    ps+1, is, k-is );
if (k=is+n-1) T->Rchild = NULL;
else  CrtBT(T->Rchild, pre[], ino[],
                        ps+1+(k-is), k+1, n-(k-is)-1 )
```



5.删除并释放二叉树中以元素值为x的结点作为根的各子树

整个过程分为两个方面：

编写释放子树空间算法

遍历中查找元素值为*x*的结点

查到该结点时，调用算法释放子树空间。

释放二叉树的所有结点空间

·二叉树为空时，不必释放；

·若T不为空，则先释放其左右子树的所有结点的空间，再释放根结点的空间——后序。

若在释放子树的空间前，先释放根结点的空间，则需要将子树的根结点的指针暂存到其他变量；否则，无法找到子树。

```
void deleteBiTree(BiTree &T){    // 此处T应为引用参数
    if ( T != NULL ){
        deleteBiTree(T->lchild );
        deleteBiTree(T->rchild );
        // 访问结点->释放结点的空间
        free(T);
        T = NULL;
    }
}
```

删除并释放二叉树中以元素值为x的结点作为根的各子树

```
void deleteXTree(BiTree &T, ElemType x)
{
    // 基于先序的查找
    if ( T != NULL ){
        // 访问结点->判断是否为指定结点->释放树空间
        if ( T->data== x ) deleteBiTree(T);
        else{
            // 此处else不能省略
            deleteXTree(T->lchild, x);
            deleteXTree(T->rchild, x);
        }
    }
}
```

6 求位于二叉树先序序列中第k个位置的结点的值

【本例特征】

如何处理多个返回结果？

【思路】

·待查结点的存在性：

当二叉树为空树，或者k非法时，待查结点不存在

➔ 函数应返回待查结点是否存在的状态指示：TRUE或FALSE

·当待查找的结点存在时，需进一步返回该结点的值

问题1：该算法需要返回多个值，如何处理？

答：一种做法是用返回值返回存在性，用变参返回值。

问题2：该算法可以基于二叉树的先序遍历的递归算法来构造，如何知道当前访问的结点是先序序列中的第几个结点？

答：引入计数器，对于该计数器可以采用全局变量来存储，也可以通过变参来处理。

求位于二叉树先序序列中第k个位置的结点的值

【算法】

```
Status PreorderKnode(BiTree T, int k, ElemType &e, int &count){
 // 输入：T为二叉链表示的二叉树，k为待查找的结点在先序序列中的位序
 // 输出：TRUE：待查找的结点存在；FALSE：待查找的结点不存在
 //       e—当待查结点存在时，该结点的值通过e带回
 //       中间变量：count—记录当前已经访问过的结点个数
   if ( T==NULL) return FALSE;
   count++;                              // 访问结点→对已访问的结点进行计数
   if (count==k ){                       //       →判断该结点是否是待查找的结点
      e = T->data;
      return TRUE;          // 查到，则设置e，并返回TRUE
   }else if (count > k)
      return FALSE; // 计数器count已经超出k(当k<0时)，则直接返回FALSE
   else {
      if (PreorderKnode(T->lchild, k, e, count)==FALSE) // 在左子树中查找
         return PreorderKnode(T->rchild, k, e, count);  // 在右子树中查找
      return TRUE;
   }
}
```

层次遍历算法及其应用

- 层次遍历算法及其应用
  - 二叉树的层次遍历算法
    - 先访问的结点，其孩子结点必然也先访问——队列
  - 基于层次遍历算法的应用
    - 求二叉树的最大宽度
    - 判断一棵二叉树是否为完全二叉树
    - 找出距指定结点最近或最远的叶子子孙
    - 找出指定层中含有的叶子/度为2/度为1的结点

二叉树的层次遍历(1)

【思路】

先访问的结点，其孩子结点必然也先访问。引入队列存储已被访问的、但其左右孩子尚未访问的结点的指针。若使用链队列，其元素可定义为：

```
typedef struct QNode{
    Bitree          data;          // 指向二叉树结点的指针
    struct QNode    *next;
} QNode, *QueuePtr;
```

【算法】

```
StatusLevelTraverse (BiTree T, Status ( *Visit ) (ElemType e) )
{
    // 初始化队列，队列的元素为二叉树的结点指针
    InitQueue(Q);
    if ( T != NULL ){
        // 访问根
        if ( !Visit( T->data ) )  return ERROR;
        // EnQueue( )为入队函数，T为待入队的元素
        EnQueue(Q, T);
```

二叉树的层次遍历(2)

```
// 从队列中取已被访问的、但其左右孩子尚未访问的结点指针
// 访问其孩子
while( !QueueEmpty(Q) ){ // 队不为空,则尚有未访问其孩子的结点
    // DeQueue( )为出队函数，它将原队头元素作为返回值返回
    p = DeQueue (Q); // 出队的是已访问结点
    // 访问左孩子
    if ( p->lchild != NULL ) {
        if ( !Visit( p->lchild ->data ))     return ERROR;
        EnQueue(Q, p->lchild );
    }
    // 访问右孩子
    if( p->rchild != NULL ) {
        if ( !Visit( p->rchild ->data ))     return ERROR;
        EnQueue(Q, p->rchild );
    }
}
    }
    return OK;
}
```

求二叉树的最大宽度

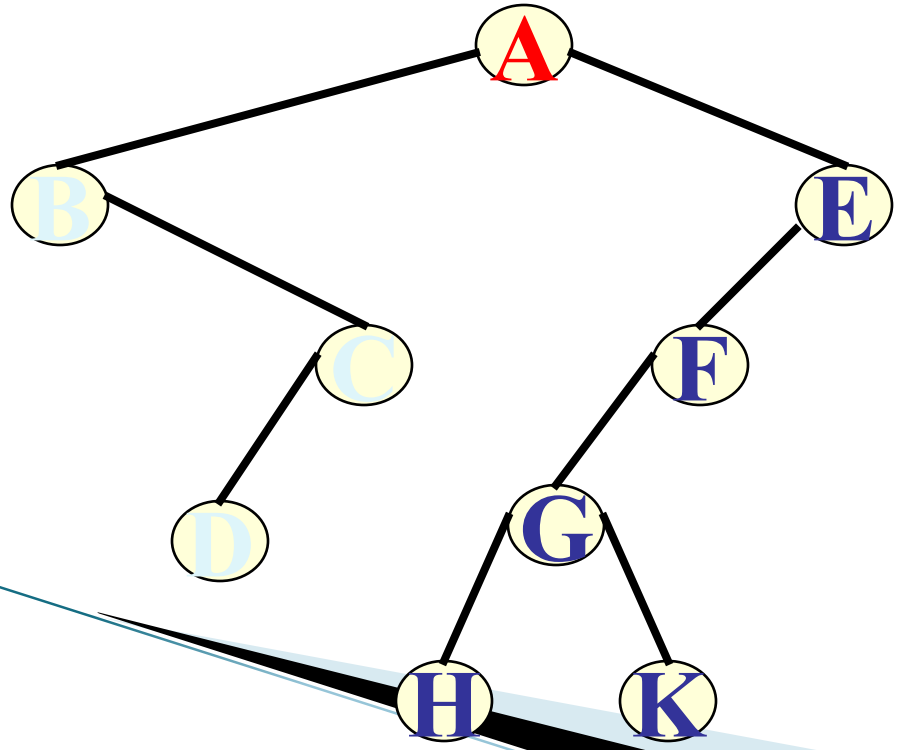
```
int width(BitTree bt)
{ if (bt==null)return 0;
   else {InitQueue (Q); EnQueue(Q,bt);last=1;
         temp=0;maxw=0;
         While(Q.front<last)
         {   DeQueue(Q,P);temp++;
            if(p->lchild) EnQueue(Q,p->lchild);
            if(p->rchild) EnQueue(Q,p->rchild);
            if(Q.front>=last){  last=rear;  if(temp>maxw)
                 maxw=temp;temp=0;}//if
          }//while
  return maxw;
}
```

6.5 线索二叉树

- 何谓线索二叉树?
- 线索链表的遍历算法
- 如何建立线索链表?

一、何谓线索二叉树?

遍历二叉树的结果是，
求得结点的一个线性序列。



例如：

先序序列:
A B C D E F G H K

中序序列:
B D C A H G K F E

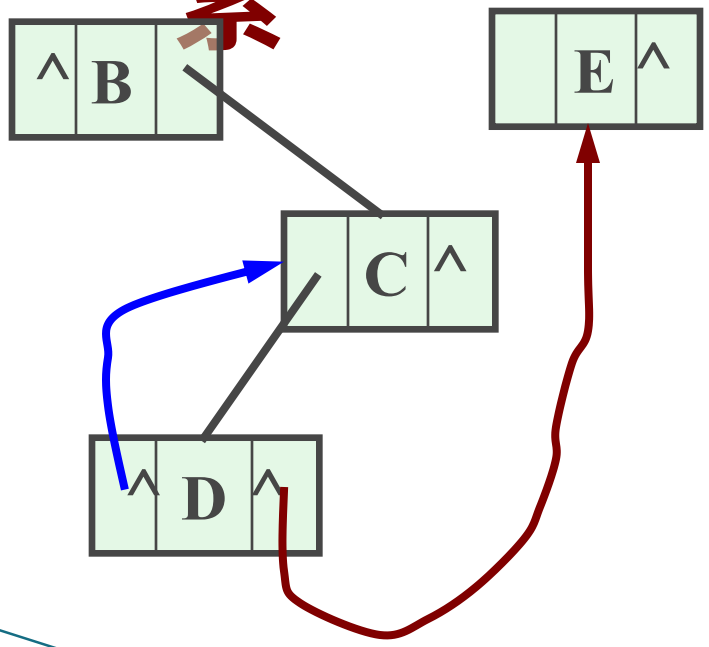
后序序列:
D C B H K G F E A

二叉树是非线性结构，从前面的讨论我们知道，以某种规则遍历二叉树时将把二叉树中的结点按该规则排列成一个线性结构，这实质上是对一个非线性结构进行线性化操作。但是，我们每次按某种规则遍历二叉树时都没有把遍历时得到的每个结点的后继结点信息和前驱结点信息保存下来，因此，我们不能像操作双向链表一样操作二叉树。

- 存这种按某种规则遍历二叉树时得到的每个结点的后继结点信息和前驱结点信息的最常用的方法是建立线索二叉树的存储结构。
- 分析二叉树结构可以发现，在有n个结点的二叉存储结构的二叉树中必定存在n+1个空链域。我们希望能利用这些空链建立起每个结点的前驱结点信息和后继结点信息。

- 指向该线性序列中的“前驱”和“后继”的指针，称作“**线索**”

A B C D E F G H K



- 包含“线索”的存储结构，称作“**线索链表**”
- 其相应的二叉树，称作“**线索二叉树**”

对线索链表中结点的约定：

在二叉链表的结点中**增加两个标志域，**并作如下规定：

- **若该结点的左子树不空，**

则Lchild域的指针指向其左子树，

且左标志域的值为“指针 Link”；

否则，Lchild域的指针指向其“前驱”

且左标志的值为“线索 Thread”。

- 若该结点的右子树不空，

则rchild域的指针指向其右子树，

且右标志域的值为 “指针 Link”；

否则，rchild域的指针指向其“后继”，

且右标志的值为“线索 Thread”。

如此定义的二叉树的存储结构称作“**线索链表**”。

线索链表的类型描述：

```
typedef enum { Link, Thread } PointerThr;
  // Link==0:指针，Thread==1:线索

typedef struct BiThrNod {
  TElemType       data;
  struct BiThrNode  *lchild, *rchild;  // 左右指针
  PointerThr        LTag, RTag;    // 左右标志
} BiThrNode, *BiThrTree;
```



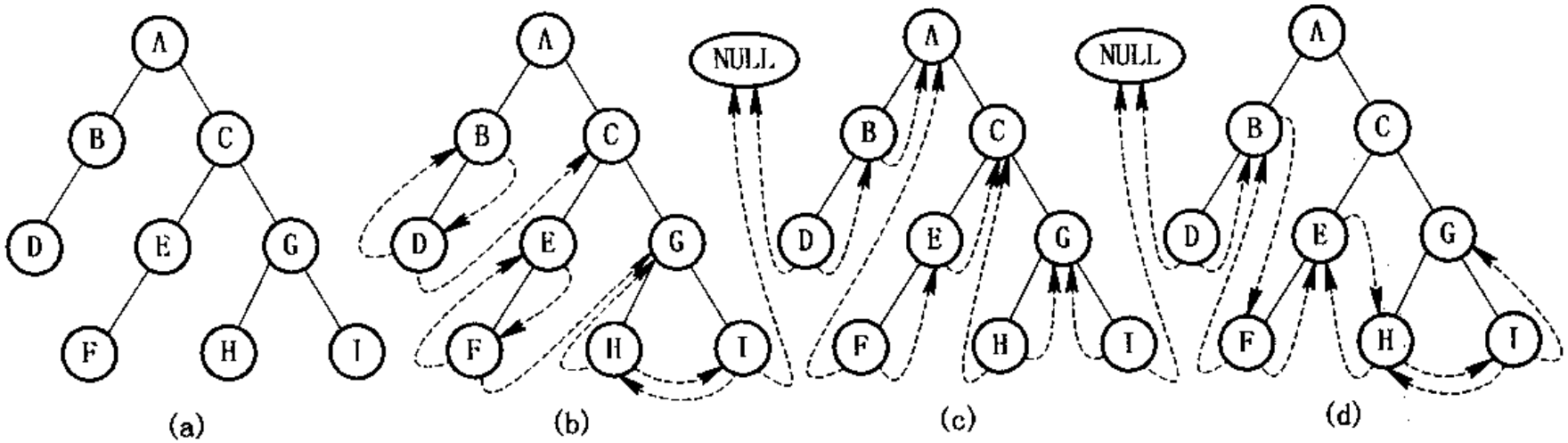
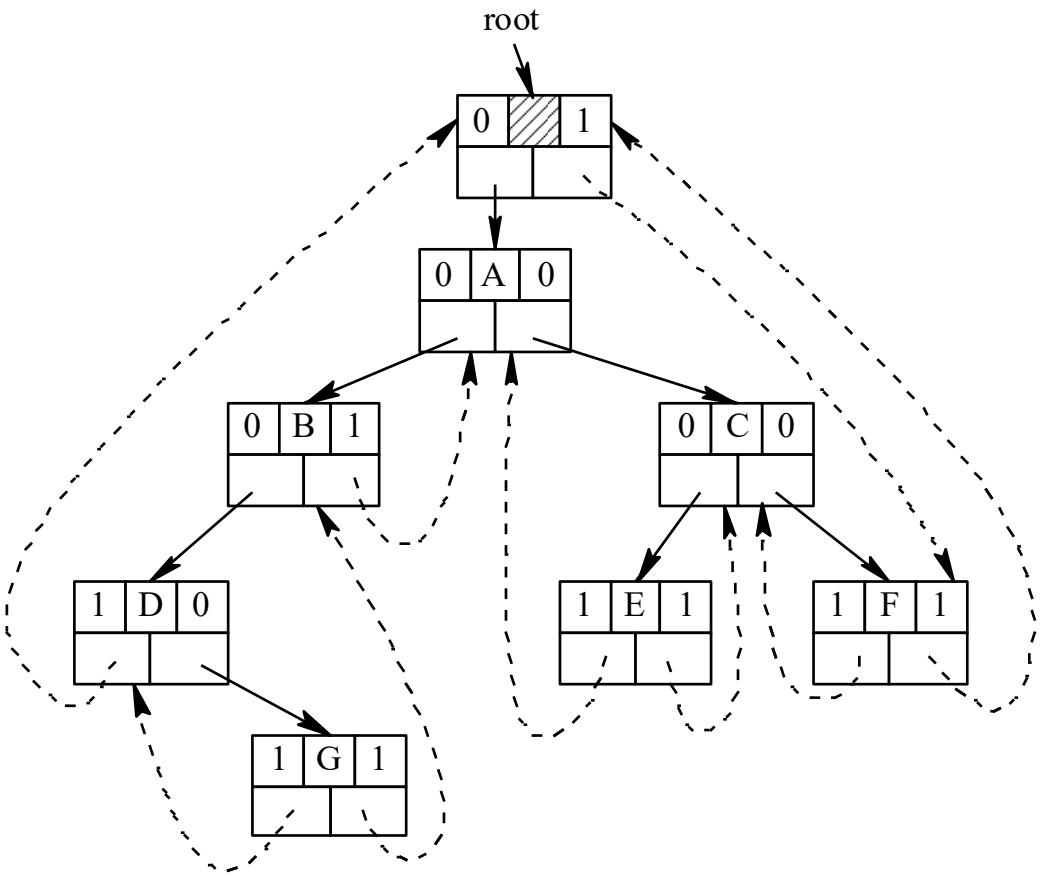


图 6.12　线索二叉树的逻辑表示图

(a) 二叉树；(b) 先根线索二叉树；(c) 中根线索二叉树；(d) 后根线索二叉树

- 模仿线性表的存储结构,在二叉树的线索链表上也添加一个头结点,令其lchild域的指针指向二叉树的根结点,其rchild域的指针指向中序遍历时访问的最后一个结点;同时,令二叉树中序序列中的第一个结点lchild域 指针的和最后一个结点rchild域的指针均指向头结点;就像为二叉树建立了一个双向线索链表, 就好比人在一个圆圈上走路,有可能存在好走的可能性

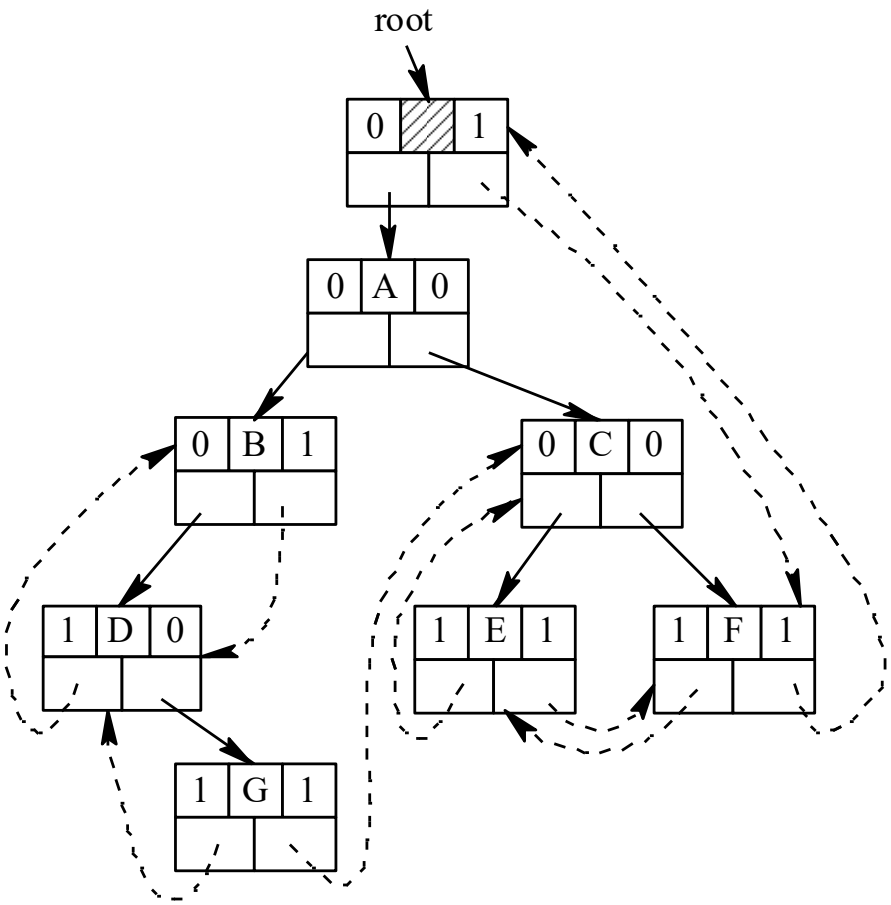


(*a*)

线索二叉树

（a）中序线索二叉树；（b）前序线索二叉树；

（c）后序线索二叉树

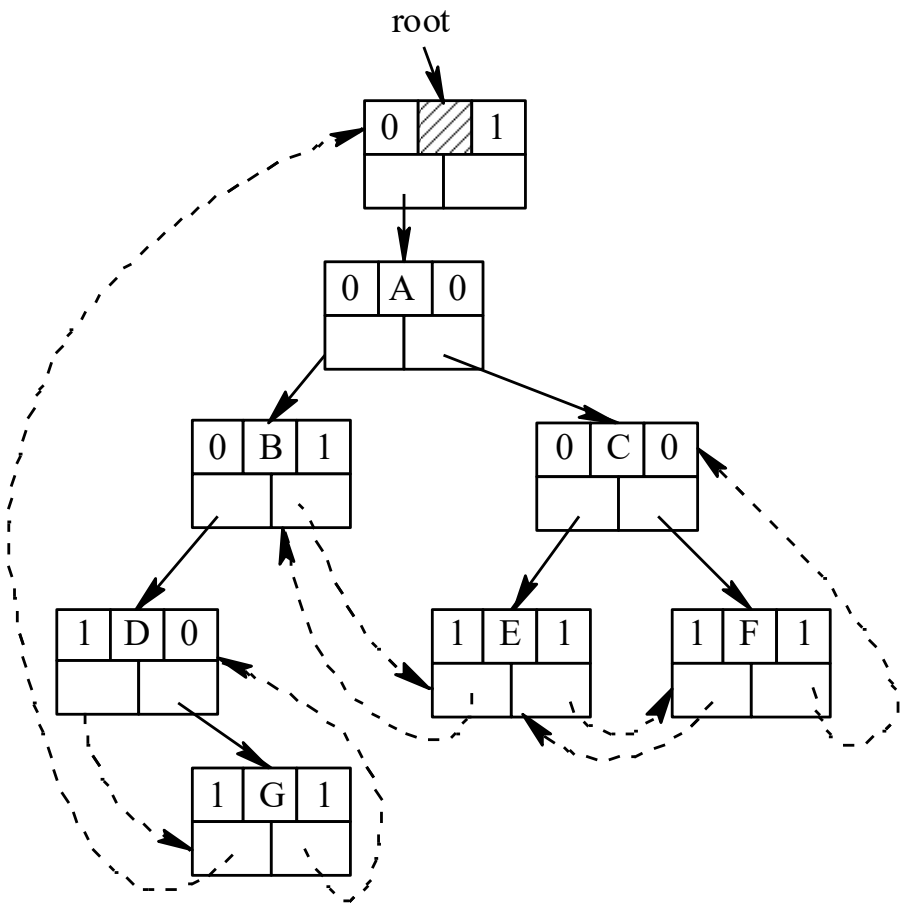


(*b*)

线索二叉树

（a）中序线索二叉树；（b）前序线索二叉树；

（c）后序线索二叉树



(*c*)

线索二叉树

（a）中序线索二叉树；（b）前序线索二叉树；

（c）后序线索二叉树

二、线索链表的遍历算法:

由于在线索链表中添加了遍历中得到的“前驱”和“后继”的信息，从而简化了遍历的算法。

```
for ( p = firstNode(T); p; p = Succ(p) )
    Visit (p);
```

例如:对中序线索化链表的遍历算法

※ 中序遍历的第一个结点 ？

左子树上处于“**最左下**”（没有左子树）的结点。

※ 在中序线索化链表中结点的后继 ？

若无右子树，**则为后继线索**所指结点；

否则为对其**右子树**进行中序**遍历**时访问的**第一个结点。**

```
void InOrderTraverse_Thr(BiThrTree T,
                         void (*Visit)(TElemType e)) {
  p = T->lchild;        // p指向根结点
  while (p != T) {      // 空树或遍历结束时，p==T
    while (p->LTag==Link)  p = p->lchild;  // 第一个结点
          Visit(p->data); // 先访问当前子树的最左结点
      while (p->RTag==Thread && p->rchild!=T) {
       p = p->rchild;  Visit(p->data);      // 访问后继结点
    }
    p = p->rchild;          // p进至其右子树根
  }} // InOrderTraverse_Thr
```



三、如何建立线索链表?

在中序遍历过程中修改结点的左、右指针域，以保存当前访问结点的“前驱”和“后继”信息。遍历过程中，附设指针pre，并始终保持指针pre指向当前访问的、指针p所指结点的前驱。

```
void InThreading(BiThrTree p) {
  if (p) {    // 对以p为根的非空二叉树进行线索化
    InThreading(p->lchild);      // 左子树线索化
    if (!p->lchild)      // 建前驱线索
      { p->LTag = Thread;    p->lchild = pre; }
    if (!pre->rchild)   // 建后继线索
      { pre->RTag = Thread;   pre->rchild = p; }
    pre = p;              // 保持 pre 指向 p 的前驱
    InThreading(p->rchild);      // 右子树线索化
  } // if
} // InThreading
```

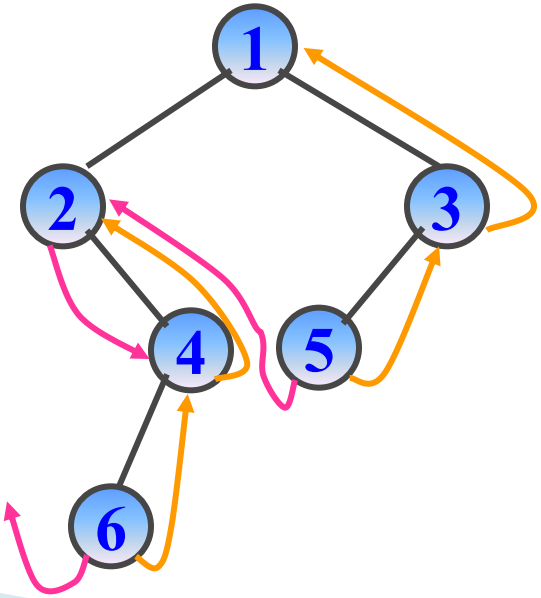
```
Status InOrderThreading(BiThrTree &Thrt,
                        BiThrTree T) { // 构建中序线索链表
  if (!(Thrt = (BiThrTree)malloc(
                                 sizeof( BiThrNode))))
    exit (OVERFLOW);
  Thrt->LTag = Link;  Thrt->RTag =Thread;
  Thrt->rchild = Thrt;      // 添加头结点
  return OK;
} // InOrderThreading
```

```
if (!T)  Thrt->lchild = Thrt;
else {
   Thrt->lchild = T;   pre = Thrt;
   InThreading(T);
   pre->rchild = Thrt;   // 处理最后一个结点
   pre->RTag = Thread;
   Thrt->rchild = pre;
}
```

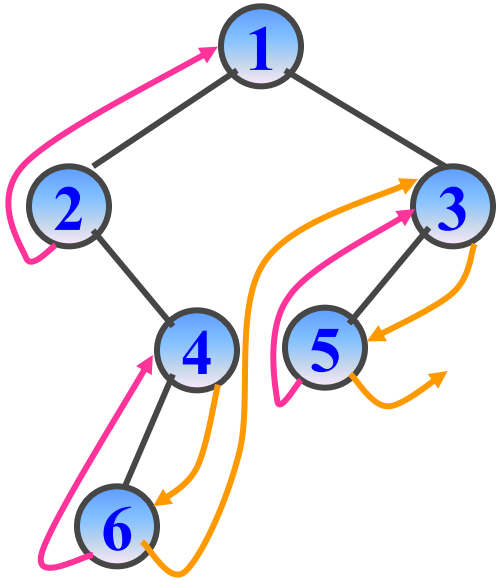


后序线索二叉树 vs 先序线索二叉树(1)

- 后序线索二叉树 与 先序线索二叉树 互为对称
  - 后序线索二叉树
    第一个结点：未必是树中最左下的结点，但一定是叶结点；
    最后一个结点：根结点



后序线索二叉树



先序线索二叉树

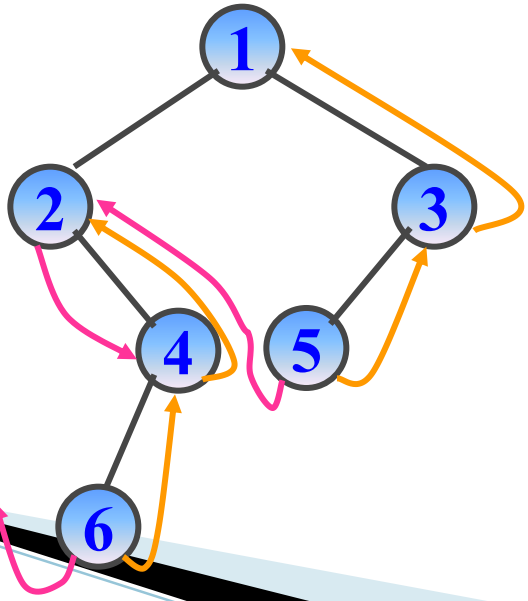
后序线索二叉树 vs 先序线索二叉树(2)

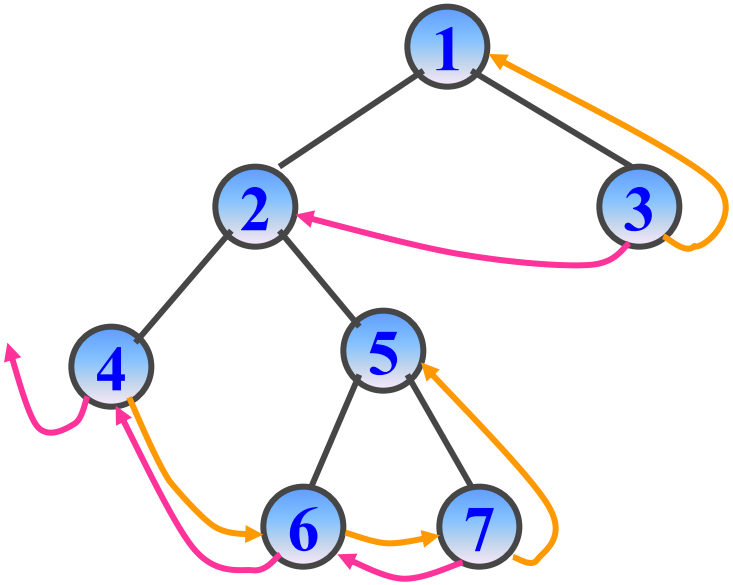
- 如何在后序线索二叉树上找结点的前驱?

a) 若该结点无左孩子，则其前驱由lchild指向。

b) 若该结点有左孩子且有右孩子，则其前驱为其右孩子;

c) 若该结点有左孩子且无右孩子，则其前驱为其左孩子。





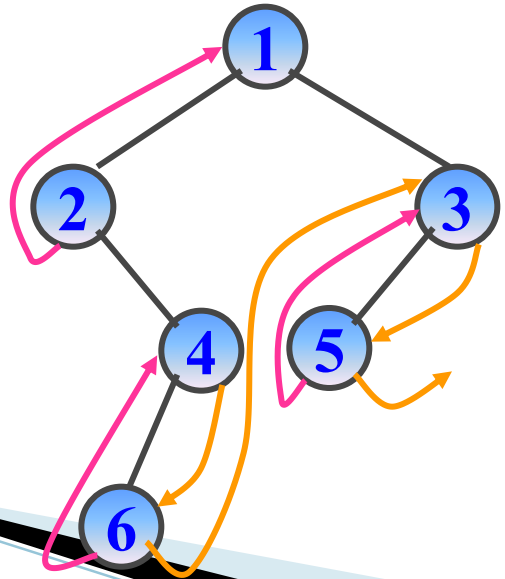
后序线索二叉树 vs 先序线索二叉树(3)

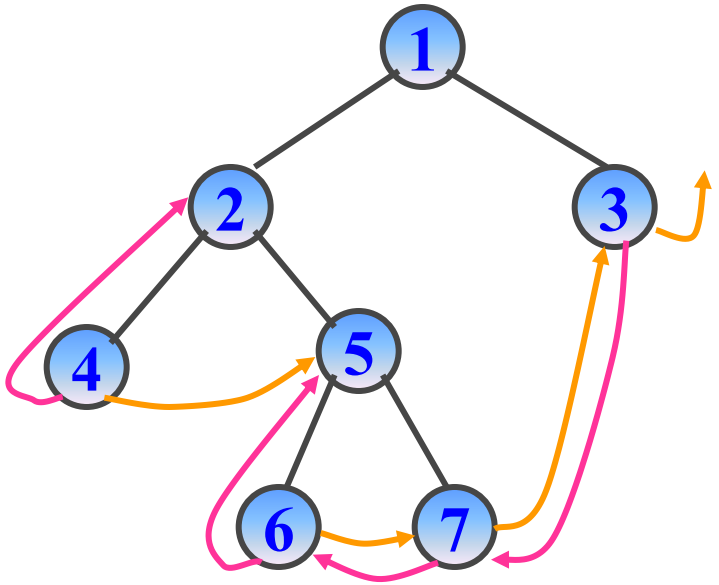
- 如何在先序线索二叉树上找结点的后继?

a) 若该结点无右孩子，则其后继由rchild指向。

b) 若该结点有右孩子且有左孩子，则其后继为其左孩子;

c) 若该结点有右孩子且无左孩子，则其后继为其右孩子。





后序线索二叉树 vs 先序线索二叉树(4)

- 如何在后序线索二叉树上找结点的后继？（复杂！☹）

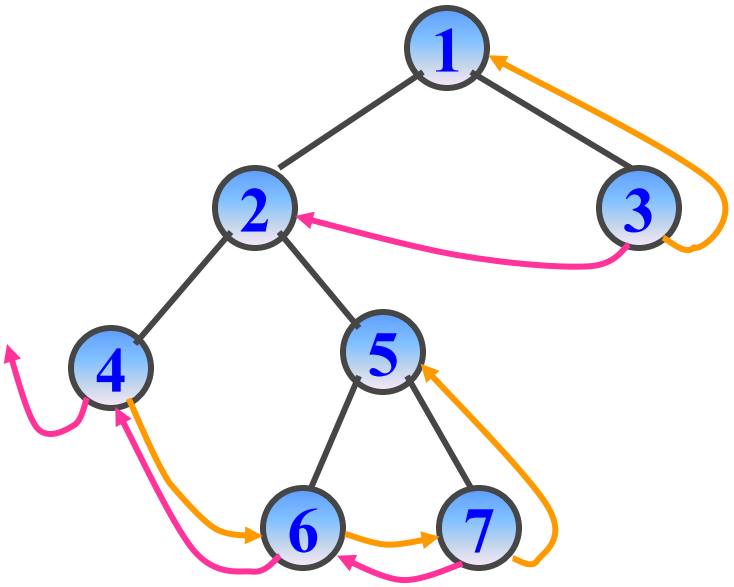
a) 若该结点是二叉树的根，则其后继为空；

b) 若该结点是其双亲的右孩子或是其双亲的左孩子且其双亲没有右子树，则其后继为其双亲；

c) 若该结点是其双亲的左孩子且其双亲有右子树，则其后继为其双亲的右子树中的后序遍历列出的第一个结点。

涉及求结点的双亲：

二叉链表→三叉链表



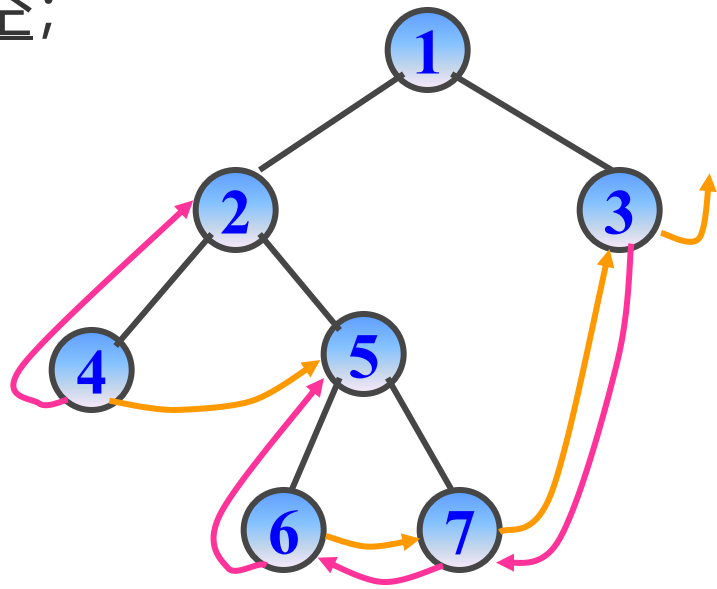
6.3 -后序线索二叉树 vs 先序线索二叉树(5)

- 如何在先序线索二叉树上找结点的前驱？(复杂！☹)

a) 若该结点是二叉树的根，则其前驱为空；

b) 若该结点是其双亲的左孩子或是其双亲的右孩子且其双亲没有左子树，则其前驱为其双亲；

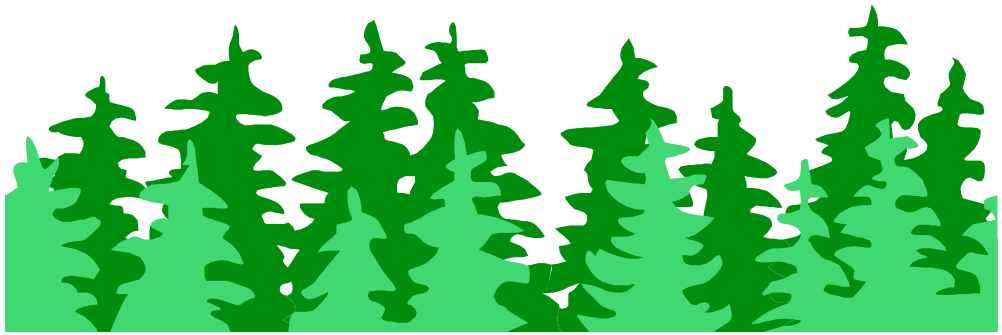
c) 若该结点是其双亲的右孩子且其双亲有左子树，则其前驱为其双亲的左子树中的先序遍历列出的最后一个结点。



涉及求结点的双亲：

二叉链表→三叉链表

6.6 树和森林的表示方法



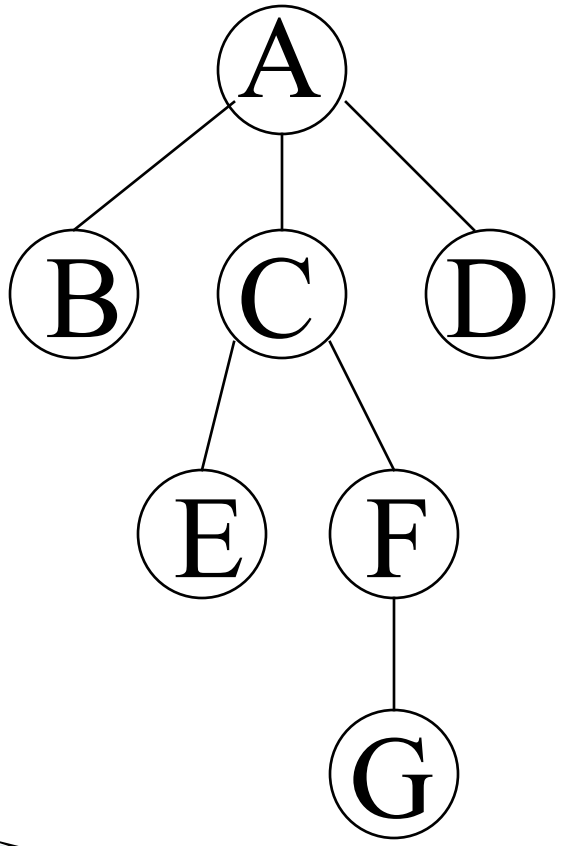
树的三种存储结构

一、双亲表示法

二、孩子链表表示法

三、树的二叉链表（孩子-兄弟）存储表示法

一、双亲表示法：



	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	2
5	F	2
6	G	5

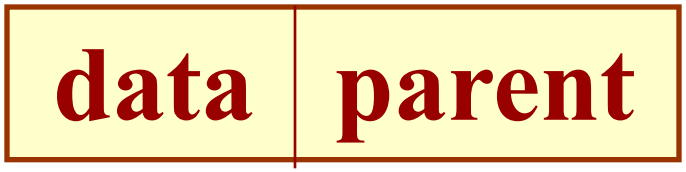
r=0

n=6

C语言的类型描述：

```
#define MAX_TREE_SIZE  100
```

结点结构：



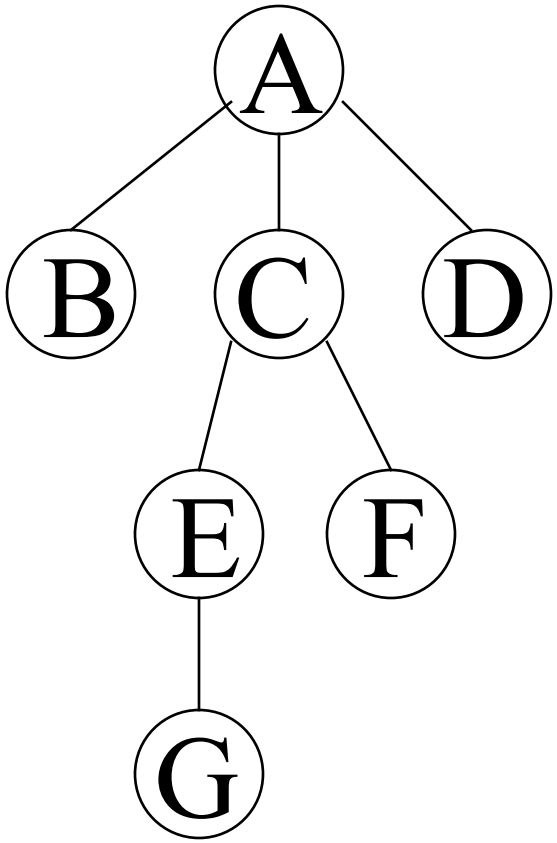

```
typedef struct PTNode {
   Elem  data;
   int    parent;   // 双亲位置域
} PTNode;
```

树结构:

```
typedef struct {
    PTNode  nodes
                    [MAX_TREE_SIZE];
    int     r, n;
            // 根结点的位置和结点个数
  } PTree;
```



二、孩子链表表示法：



data firstchild

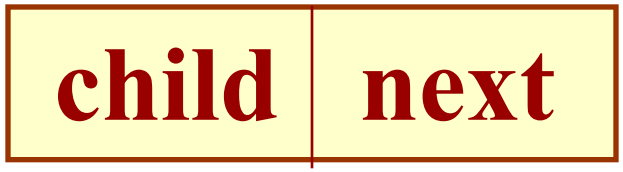
	data	firstchild	
0	A	1	→ 1 → 2 → 3 ∧
1	B	0	∧
2	C	0	→ 4 → 5 ∧
3	D	0	∧
4	E	2	→ 6 ∧
5	F	2	∧
6	G	4	∧

r=0

n=6

C语言的类型描述：

孩子结点结构：




```
typedef struct CTNode {
    int         child;
    struct CTNode *next;
} *ChildPtr;
```

双亲结点结构



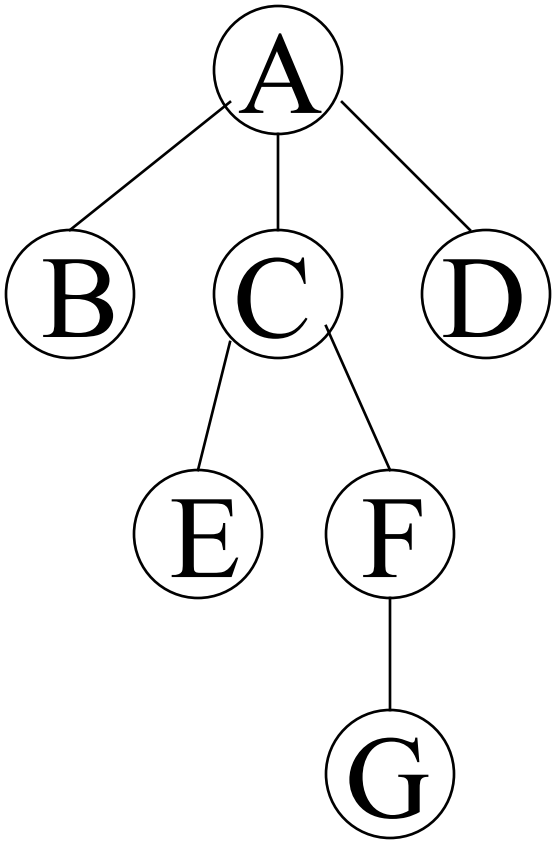

```
typedef struct {
  Elem    data;
  ChildPtr  firstchild;
                  // 孩子链的头指针
} CTBox;
```

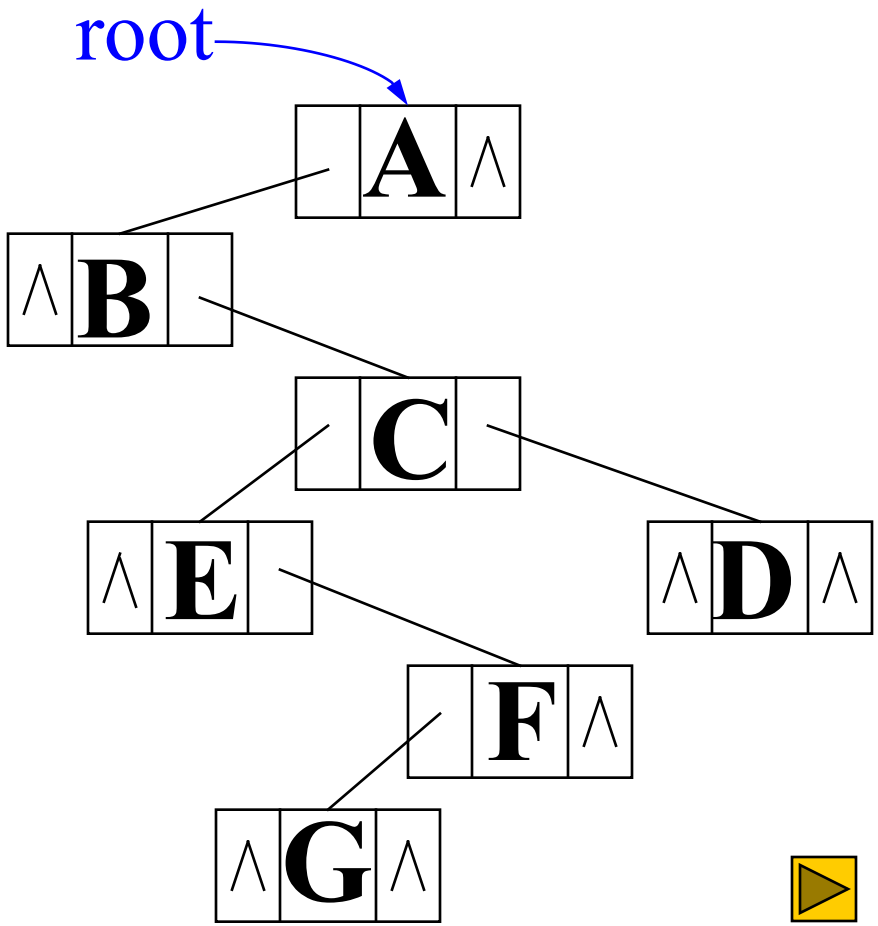
树结构:

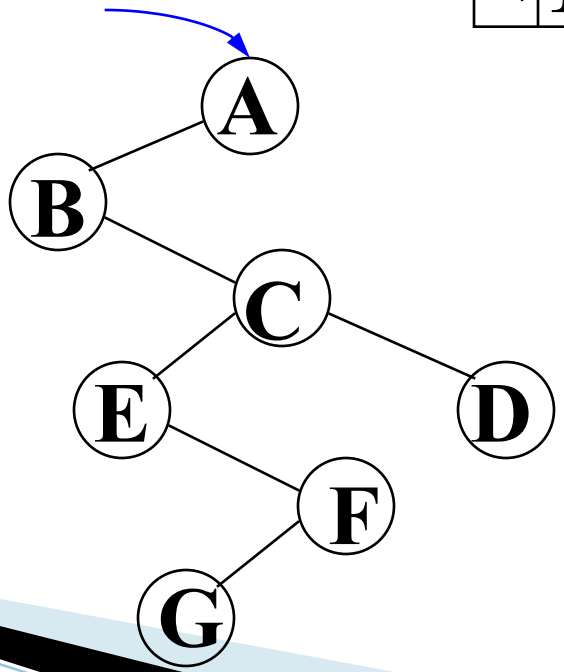
```
typedef struct {
    CTBox  nodes[MAX_TREE_SIZE];
    int    n, r;
           // 结点数和根结点的位置
} CTree;
```



三、树的二叉链表(孩子-兄弟)存储表示法







C语言的类型描述：

结点结构：




```
typedef struct CSNode{
    Elem        data;
    struct CSNode
            *firstchild, *nextsibling;
} CSNode, *CSTree;
```

森林和二叉树的对应关系

设森林

$$F = (T_1, T_2, \cdots, T_n);$$

$$T_1 = (root,\ t_{11}, t_{12}, \cdots, t_{1m});$$

二叉树

$$B = (LBT, Node(root), RBT);$$

由森林转换成二叉树的转换规则为:

若 $F = \Phi$，则 $B = \Phi$;

否则，

由 $ROOT(T_1)$ 对应得到 Node(root);

由 $(t_{11}, t_{12}, \cdots, t_{1m})$ 对应得到 LBT;

由 $(T_2, T_3, \cdots, T_n)$ 对应得到 RBT。

由二叉树转换为森林的转换规则为：

若 $B = \Phi$，则 $F = \Phi$；

否则，

由 Node(root) 对应得到 ROOT(T_1)；

由LBT 对应得到 ($t_{11}, t_{12}, \cdots, t_{1m}$)；

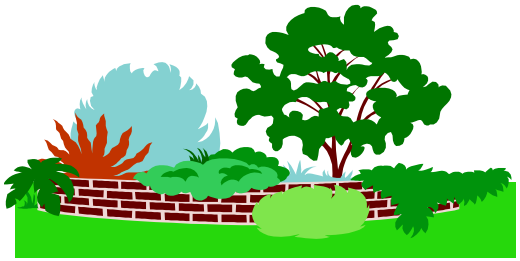
由RBT 对应得到 ($T_2, T_3, \cdots, T_n$)。

由此，树的各种操作均可对应二叉树的操作来完成。

应当注意的是，和树对应的二叉树，其左、右子树的概念已改变为：**左是孩子，右是兄弟。**

6.7

树和森林的遍历



一、树的遍历

二、森林的遍历

三、树的遍历的应用

树的遍历可有三条搜索路径:

先根(次序)遍历:

若树不空，则先访问根结点，然后依次先根遍历各棵子树。

后根(次序)遍历:

若树不空，则先依次后根遍历各棵子树，然后访问根结点。

按层次遍历:

若树不空，则自上而下自左至右访问树中每个结点。

先根遍历时顶点的访问次序：

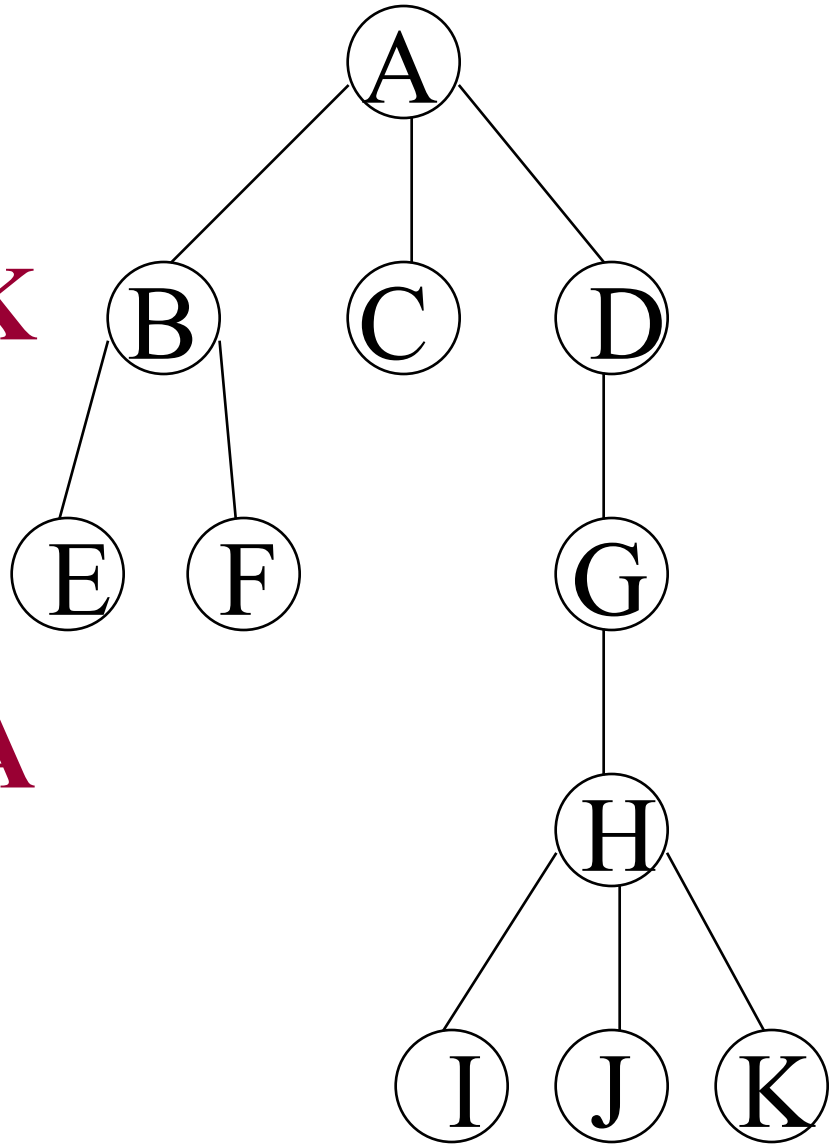
A B E F C D G H I J K

后根遍历时顶点的访问次序：

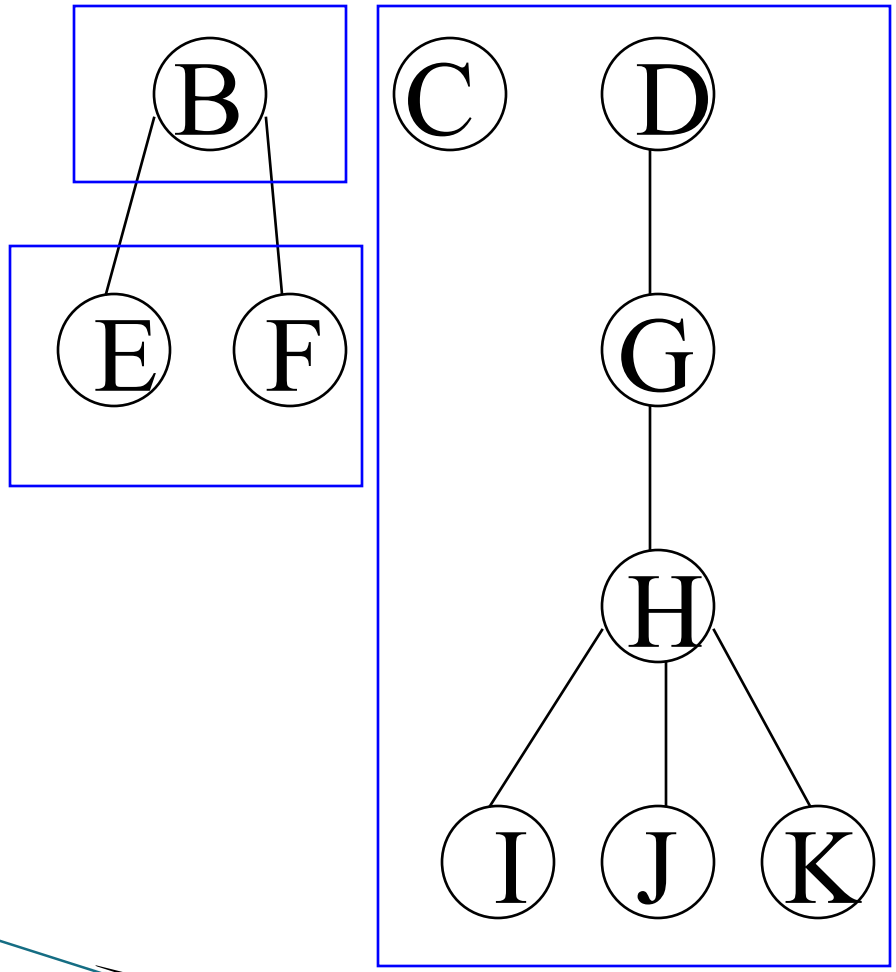
E F B C I J K H G D A

层次遍历时顶点的访问次序：

A B C D E F G H I J K



森林由三部分构成：



1．森林中第一棵树的根结点；

2．森林中第一棵树的子树森林；

3．森林中其它树构成的森林。

森林的遍历

1. **先序遍历** 若森林不空，则

访问森林中第一棵树的根结点；

先序遍历森林中第一棵树的子树森林；

先序遍历森林中(除第一棵树之外)其余树构成的森林。

即：**依次从左至右**对森林中的每一棵**树**进行**先根遍历**。

2．中序遍历

若森林不空，则

中序遍历森林中第一棵树的子树森林；

访问森林中第一棵树的根结点；

中序遍历森林中(除第一棵树之外)其余树构成的森林。

即：**依次从左至右**对森林中的每一棵**树**进行**后根遍历**。

树的遍历和二叉树遍历的对应关系 ？

树	森林	二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历	中序遍历

设树的存储结构为孩子兄弟链表

```
typedef struct CSNode{
    Elem        data;
    struct CSNode *firstchild, *nextsibling;
} CSNode, *CSTree;
```

一、求树的深度

二、输出树中所有从根到叶子的路径

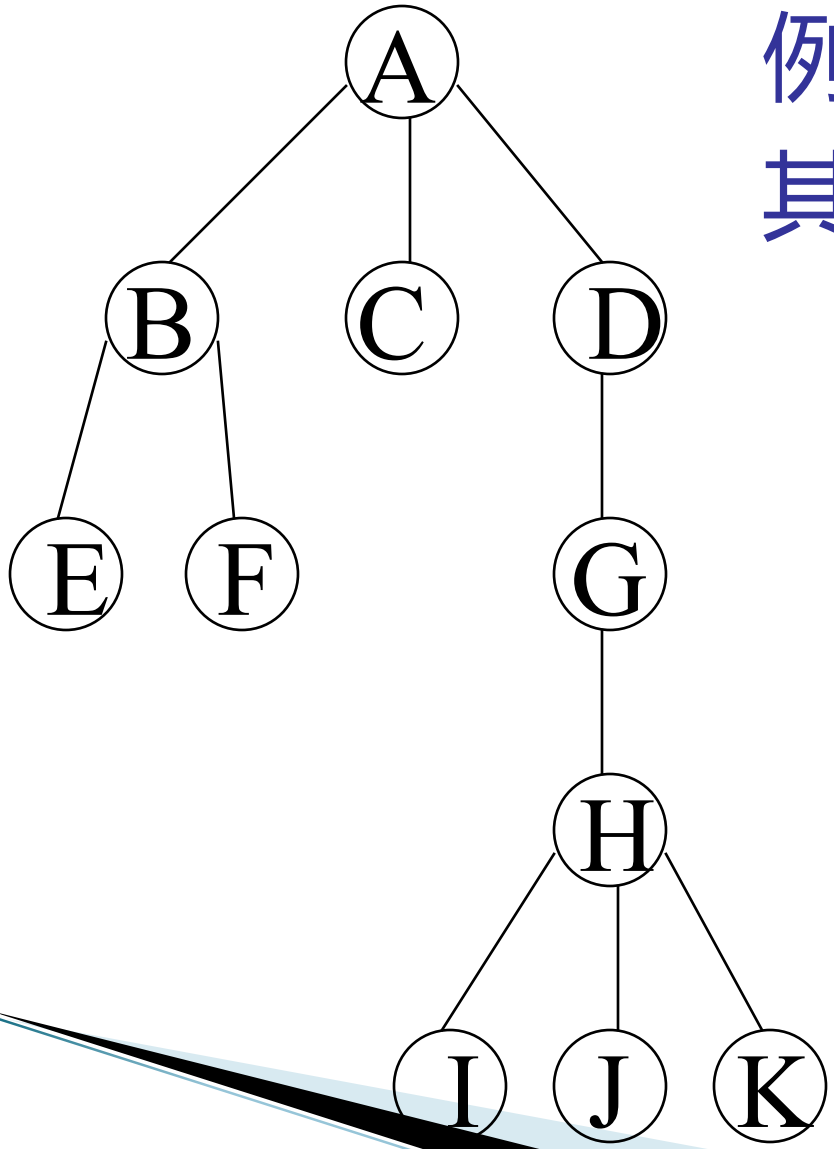
三、建树的存储结构



一、求树的深度的算法：

```
int TreeDepth(CSTree T) {
  if(!T)  return 0;
  else {
     h1 = TreeDepth( T->firstchild );
     h2 = TreeDepth( T->nextsibling);
     return(max(h1+1, h2));
  }
} // TreeDepth
```

二、输出树中所有从根到叶子的路径的算法:



例如：对左图所示的树，其输出结果应为：

A B E

A B F

A C

A D G H I

A D G H J

A D G H K

```
void AllPath( Bitree T, Stack& S ) {
 // 输出二叉树上从根到所有叶子结点的路径
  if (T) {
    Push( S, T->data );
    if (!T->Lchild && !T->Rchild ) PrintStack(S);
    else {  AllPath( T->Lchild, S );
            AllPath( T->Rchild, S );
    }
    Pop(S); // 回溯时移除当前结点
  } // if(T)
} // AllPath
```

```
void OutPath( Bitree T, Stack& S ) {
// 输出森林中所有从根到叶的路径
    if ( T ) {
      Push(S, T->data );
      if ( !T->firstchild ) Printstack(s);
      else OutPath( T->firstchild, s );
      Pop(S);
      OutPath( T->nextsibling);
    } // while
} // OutPath
```

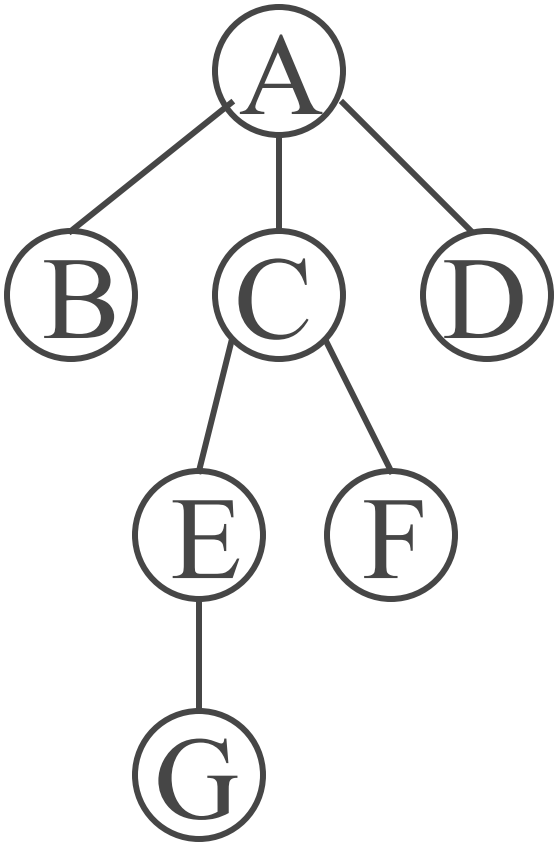


三、建树的存储结构的算法:

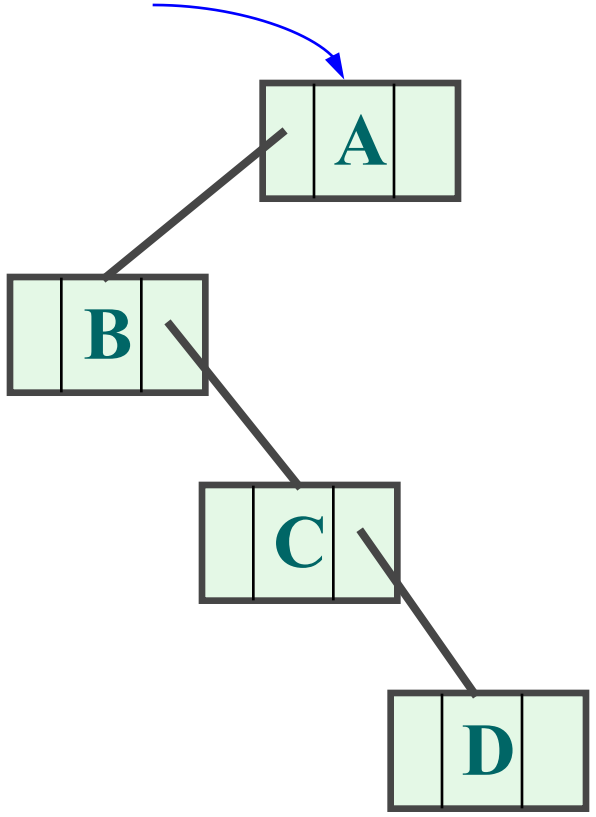
和二叉树类似，不同的定义相应有不同的算法。

假设以二元组(F，C)的形式**自上而下、自左而右**依次输入树的各边，建立树的**孩子-兄弟链表**。

例如：　对下列所示树的输入序列应为:



(‘#’, ‘A’)
(‘A’, ‘B’)
(‘A’, ‘C’)
(‘A’, ‘D’)
(‘C’, ‘E’)
(‘C’, ‘F’)
(‘E’, ‘G’)



可见，算法中需要一个**队列**保存已建好的结点的**指针**。

```
void CreatTree( CSTree &T ) {
  T = NULL;
  for( scanf(&fa, &ch); ch!=' ';
                        scanf(&fa, &ch);) {
    p = GetTreeNode(ch);    // 创建结点
    EnQueue(Q, p);          // 指针入队列
    if (fa == ' ')  T = p;  // 所建为根结点
    else { …  …  }          // 非根结点的情况
  } // for
} // CreateTree
```



```
    GetHead(Q,s);  // 取队列头元素(指针值)
while (s->data != fa ) { // 查询双亲结点
    DeQueue(Q,s);  GetHead(Q,s);
}
if (!(s->firstchild))
   { s->firstchild = p; r = p; }
                        // 链接第一个孩子结点
else { r->nextsibling = p;  r = p;  }
                          // 链接其它孩子结点
```



6.8 哈夫曼树与哈夫曼编码

- 最优树的定义
- 如何构造最优树
- 前缀编码

一、最优树的定义

- **结点的路径长度**定义为：
  从根结点到该结点的路径上分支的数目。

- **树的路径长度**定义为：
  树中每个结点的路径长度之和。

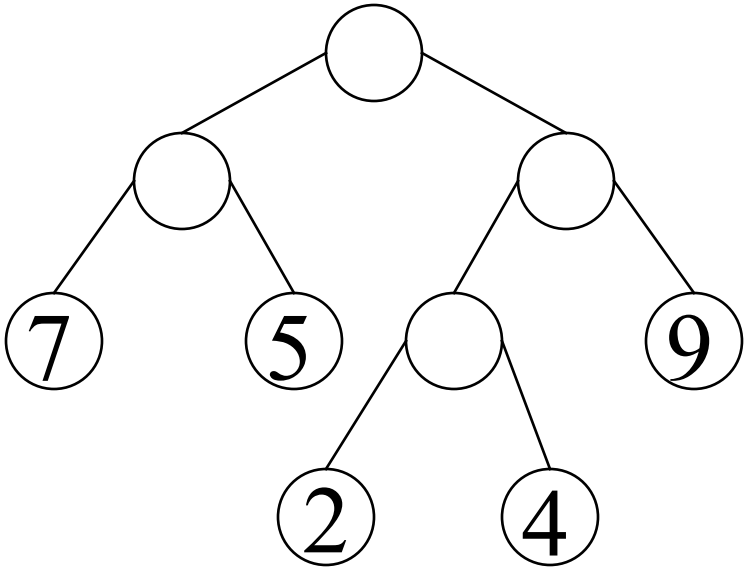
树的带权路径长度定义为：

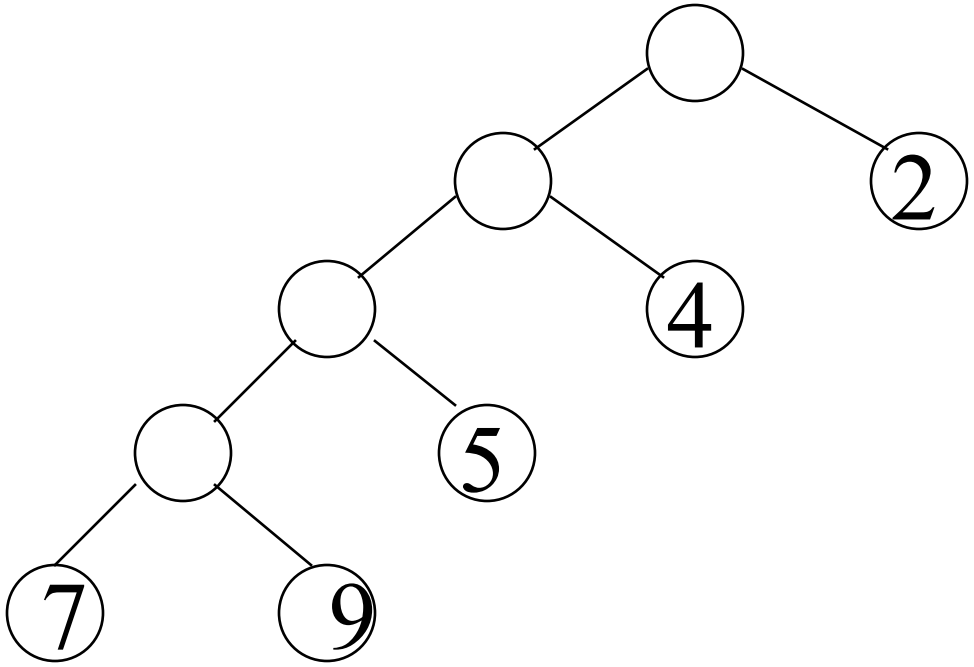
树中所有叶子**结点的带权路径长度**之和

$WPL(T) = \Sigma w_k l_k$ (对所有叶子结点)。

例如：

在所有含 n 个叶子结点、并带相同权值的 m 叉树中，必存在一棵其**带权路径长度取最小值**的树，称为“**最优树**”。





WPL(T)=

7×2+5×2+2×3+

4×3+9×2

=60

WPL(T)=

7×4+9×4+5×3+

4×2+2×1

=89

二、如何构造最优树

(赫夫曼算法) 以二叉树为例:

(1) 根据给定的 n 个权值 $\{w_1, w_2, \ldots, w_n\}$，构造 n 棵二叉树的集合

$$F = \{T_1, T_2, \cdots, T_n\},$$

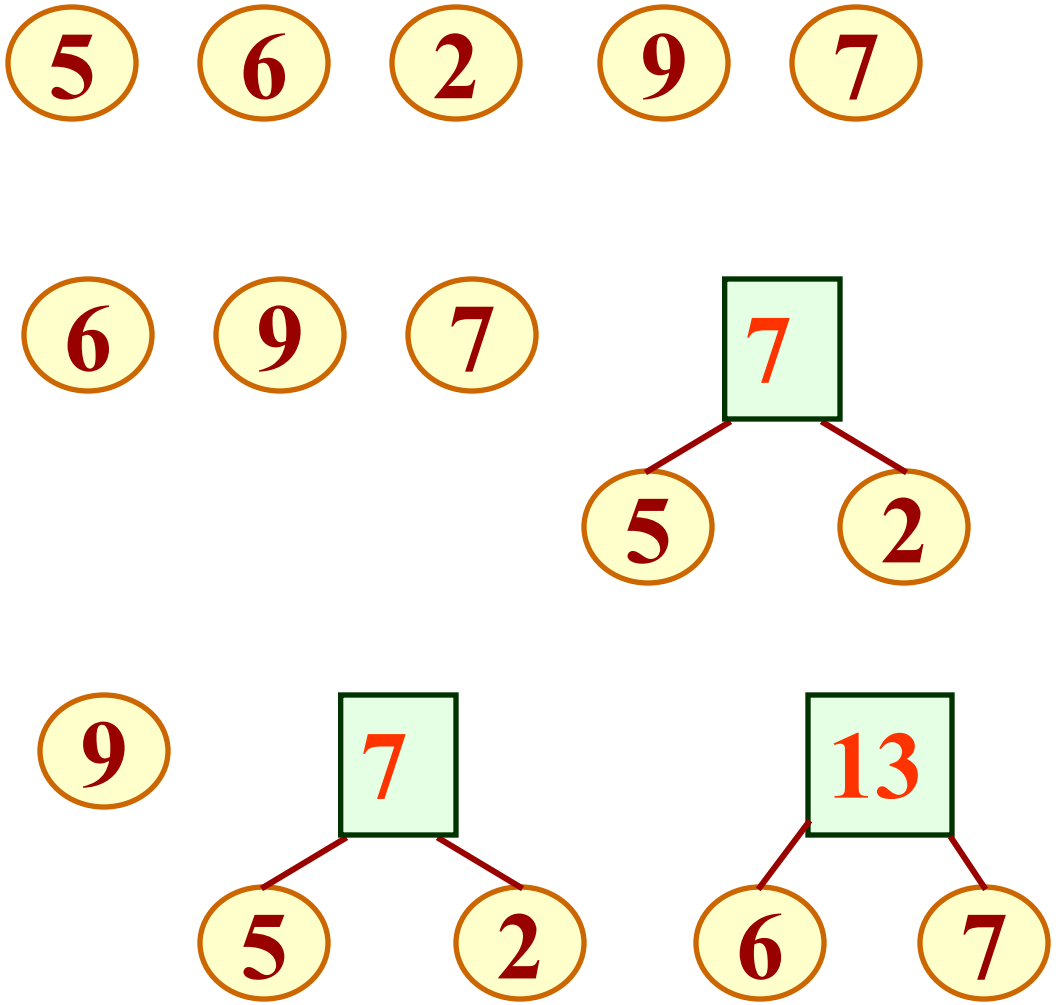
其中每棵二叉树中均只含一个带权值为 w_i 的根结点，其左、右子树为空树;

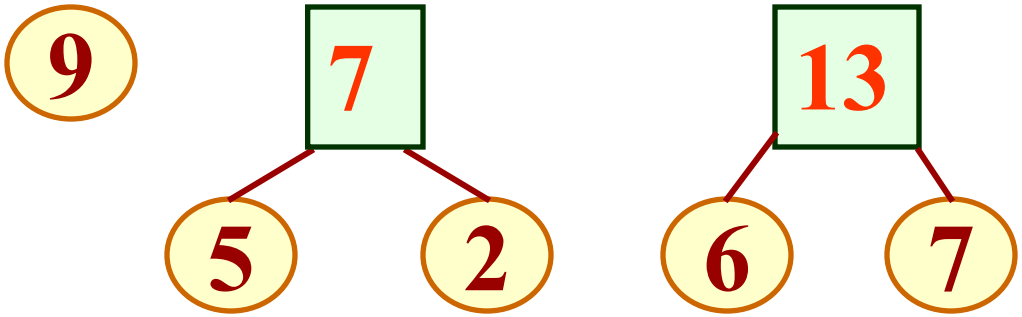
(2) 在 F 中选取其根结点的权值为最小的两棵二叉树，分别作为左、右子树构造一棵新的二叉树，并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和；

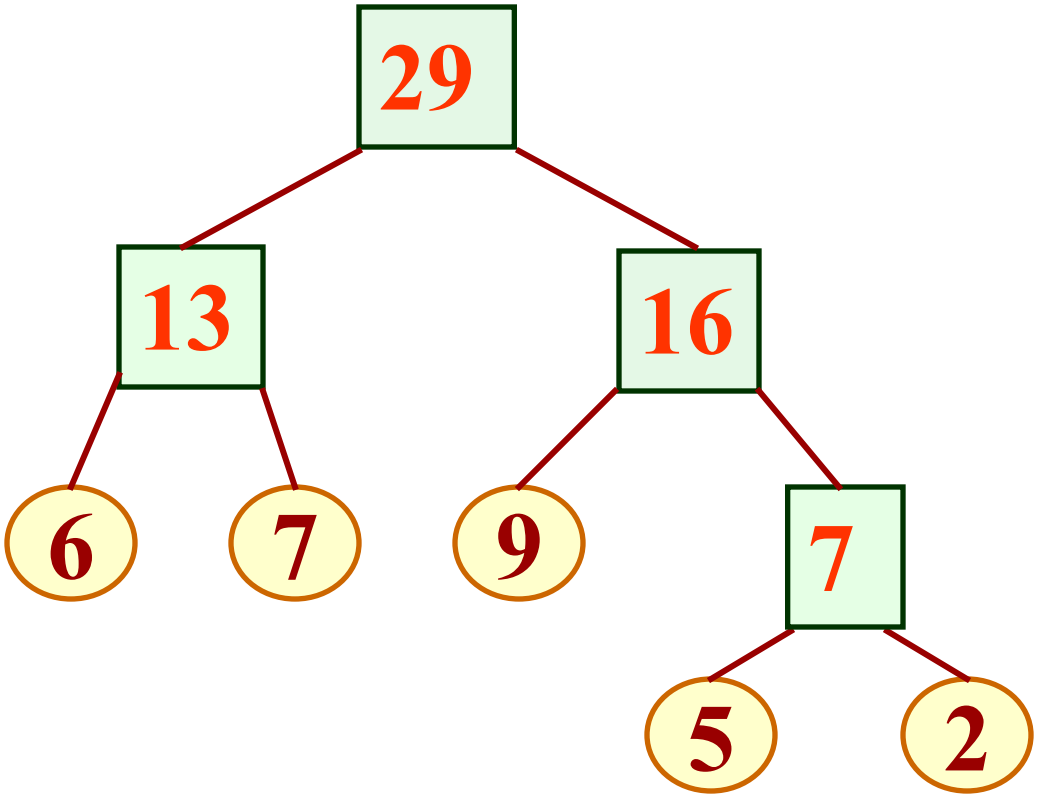
(3) 从F中删去这两棵树，同时加入刚生成的新树；

(4) 重复 (2) 和 (3) 两步，直至 F 中只含一棵树为止。

例如: 已知权值 W={ 5, 6, 2, 9, 7 }




9
7
5
2
13
6
7


29
13
16
6
7
9
7
5
2

哈夫曼算法的存储结构

1. 设置一个数组huffTree[2n−1]保存哈夫曼树中各点的信息，数组元素的结点结构 。

weight	lchild	rchild	parent

其中：weight：权值域，保存该结点的权值；

lchild：指针域，结点的左孩子结点在数组中的下标；

rchild：指针域，结点的右孩子结点在数组中的下标；

parent：指针域，该结点的双亲结点在数组中的下标。

哈夫曼伪代码

1. 数组huffTree初始化，所有元素结点的双亲、左右孩子都置为-1；
2. 数组huffTree的前n个元素的权值置给定值w[n]；
3. 进行n-1次合并
   3.1 在二叉树集合中选取两个权值最小的根结点，其下标分别为i_1, i_2；
   3.2 将二叉树i_1、i_2合并为一棵新的二叉树k；

哈夫曼树构造

第 167–168 页已有结点结构和合并伪代码。补充从未合并的根中选取最小权值结点，以及建立双亲与左右孩子关系的代码。沿用课件的 0 起始下标和 -1 空链标记。调用方分配 2*n-1 个结点，权值非负且总和可由 long long 表示。

```
#include <limits.h>

typedef struct {
    long long weight;
    int lchild, rchild, parent;
} HuffNode;

static int smallest_root(const HuffNode tree[], int end, int skip)
{
    int best = -1;
    for (int i = 0; i < end; ++i) {
        if (i != skip && tree[i].parent == -1 &&
            (best == -1 || tree[i].weight < tree[best].weight)) {
            best = i;
        }
    }
    return best;
}
```

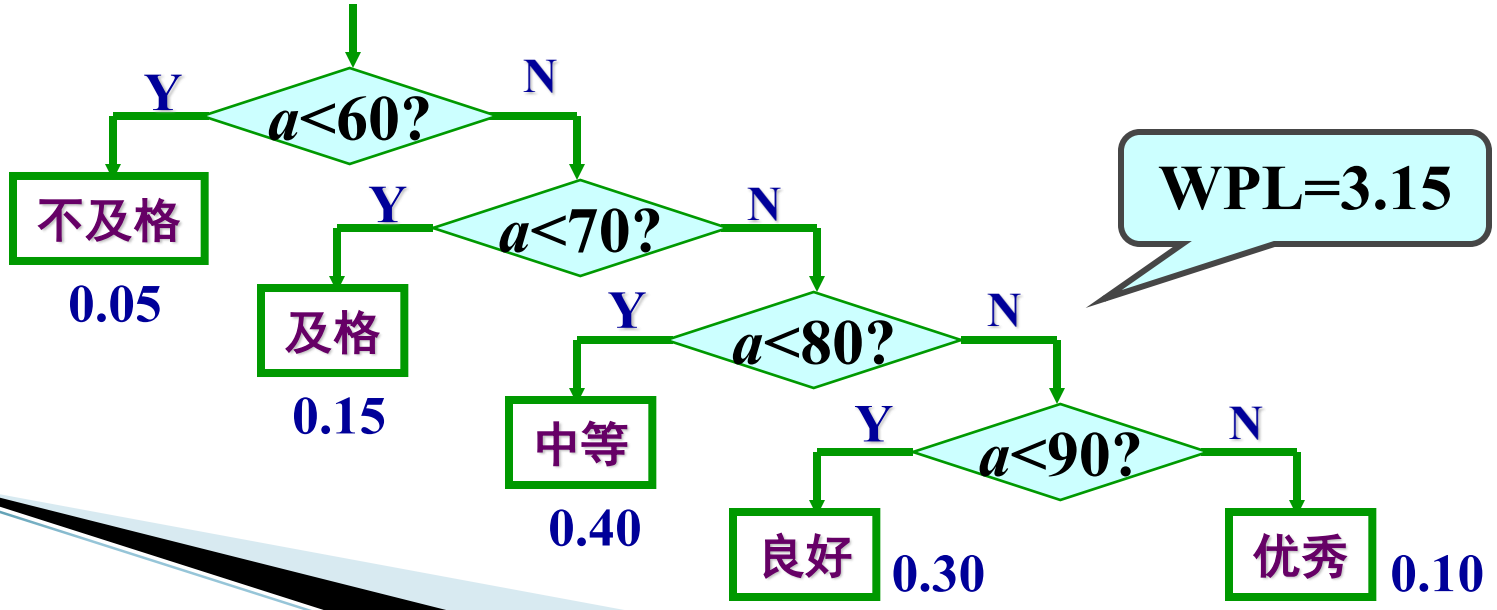
哈夫曼树构造

```
int build_huffman(const long long weights[], int n, HuffNode tree[])
{
    if (n <= 0 || n > INT_MAX / 2 + 1) {
        return -1;
    }
    int node_count = n + (n - 1);
    for (int i = 0; i < n; ++i) {
        tree[i] = (HuffNode){weights[i], -1, -1, -1};
    }
    for (int k = n; k < node_count; ++k) {
        int a = smallest_root(tree, k, -1);
        int b = smallest_root(tree, k, a);
        tree[k] = (HuffNode){tree[a].weight + tree[b].weight, a, b, -1};
        tree[a].parent = tree[b].parent = k;  // 两棵子树并入新根
    }
    return node_count - 1;
}
```

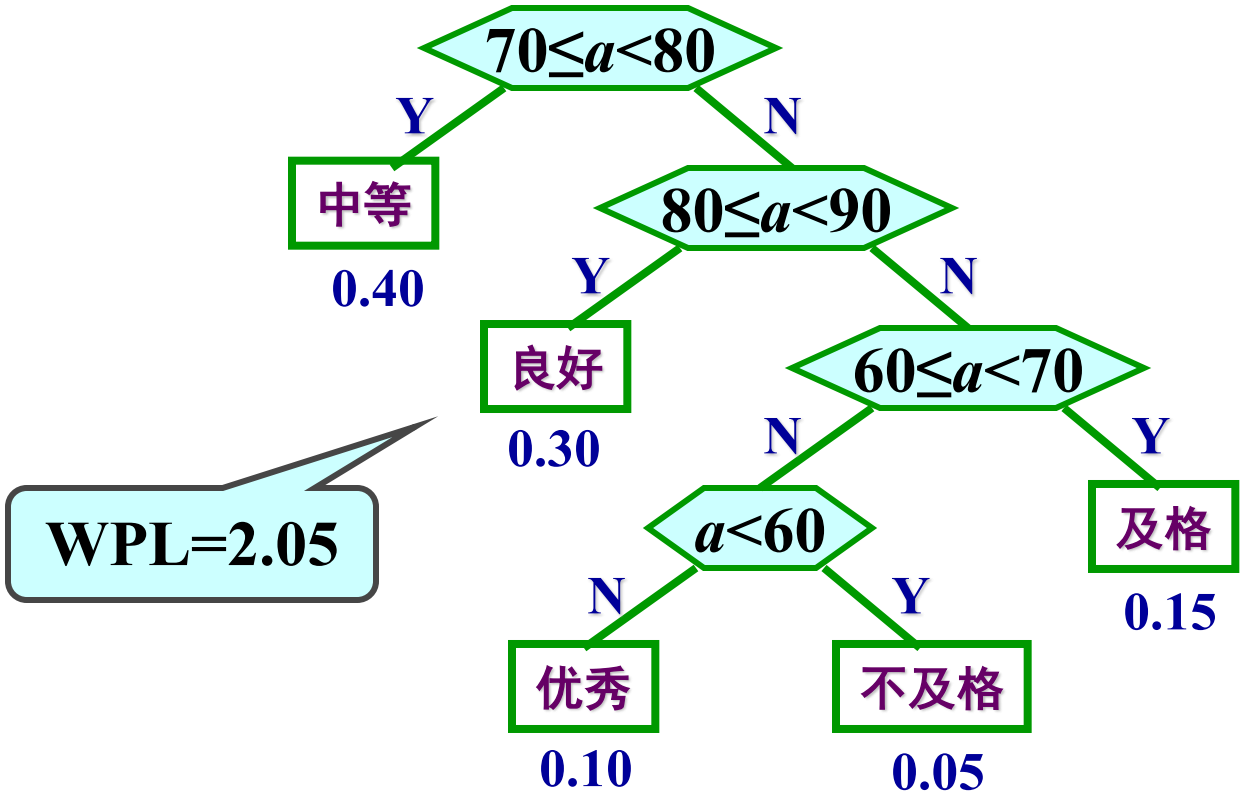
6.6 -赫夫曼树的应用(1)

- 赫夫曼树的应用——最佳判定算法

分数	0~59	60~69	70~79	80~89	90~100
比例	0.05	0.15	0.40	0.30	0.10
等级	不及格	及格	中等	良好	优秀



赫夫曼树的应用——最佳判定算法(2)



哈夫曼树应用——哈夫曼编码

- 问题描述：已知电文中出现的一组字符及出现频率，试为该组字符编码以使得电文总长最短。
- 前缀编码：某字符的编码不是其他字符编码的前缀。
- 赫夫曼编码过程：

  1) 由n个权值，形成包含有$2n-1$个结点的赫夫曼树

  2) 约定赫夫曼树的左分支表示字符‘0’，右分支表示字符‘1’，则从根结点到叶子结点的路径上的分支字符组成的字符串作为该叶子结点字符的编码。



哈夫曼树应用——哈夫曼编码

- 赫夫曼编码
  - 赫夫曼树和赫夫曼编码的存储结构选取
    - 由权值数n唯一确定树中的结点数为 $(2n - 1)$
      →一次性分配结点空间
    - 编码：从叶子到根的路径
      →快速访问双亲
    - 译码：从根到叶子的路径
      →快速访问孩子

赫夫曼树：
三叉静态链表

赫夫曼编码: 按实际长度动态分配空间

需要一个求编码的工作空间,长度为n



哈夫曼树应用——哈夫曼编码

编码方案：

A：00

B：10

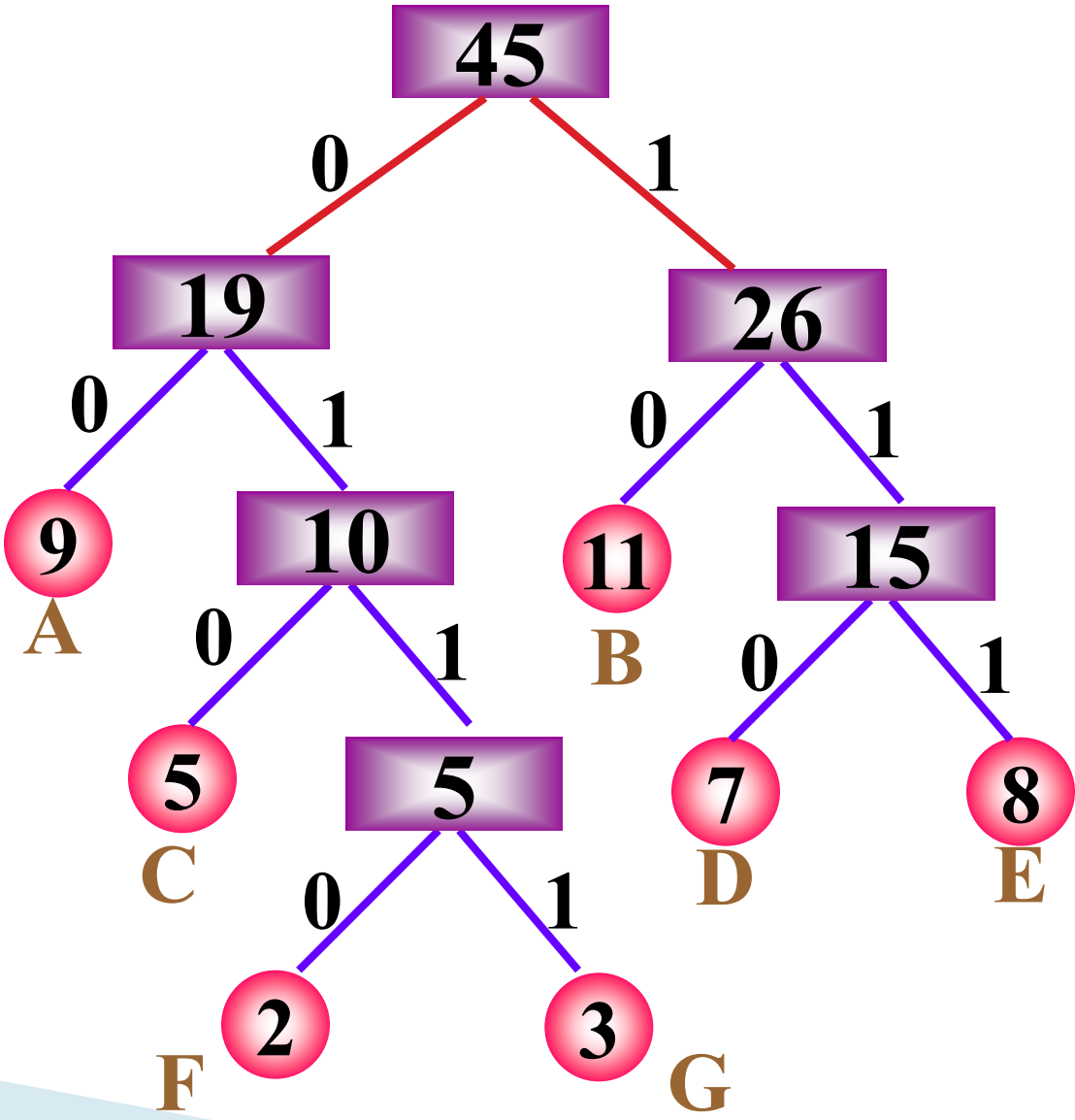
C：010

D：110

E：111

F：0110

G：0111



1. 熟练掌握**二叉树的结构特性**，了解相应的证明方法。

2. 熟悉二叉树的各种**存储结构**的特点及适用范围。

3. **遍历二叉树**是二叉树各种操作的基础。实现二叉树遍历的具体算法与所采用的存储结构有关。掌握各种遍历策略的**递归算法**，**灵活运用遍历算法**实现二叉树的其它操作。**层次遍历**是按另一种搜索策略进行的遍历。

4. 理解二叉树**线索化的实质**是建立结点与其在相应序列中的前驱或后继之间的直接联系，熟练掌握二叉树的**线索化过程**以及在中序线索化树上找给定结点的前驱和后继的方法。二叉树的**线索化过程**是**基于**对二叉树进行**遍历**，而线索二叉树上的**线索又为相应的遍历提供了方便。**

5. 熟悉**树的**各种**存储结构**及其特点，掌握**树和森林与二叉树的转换**方法。建立存储结构是进行其它操作的前提，因此读者应**掌握** 1 至 2 种**建立**二叉树和树的**存储结构的方法。**

6. 学会编写**实现树的各种操作**的算法。

7. 了解**最优树的特性**，掌握**建立最优树和哈夫曼编码**的方法。